SIMATS ENGINEERING

Design and Analysis of Algorithms

Assignment 4 – Part II

TASK 1 - Word Wrap as a Document-Layout Engine (Dynamic Programming)

> **Problem** Arrange n words into lines of width W (single space between words). A non-final line with s unused spaces costs s^2; the final line costs 0. Minimise the total penalty and report the layout, not just the number.

1.1 Implementation

Model. Let dp[i] be the minimum penalty for laying out the suffix words[i..n-1]. Every layout of that suffix starts with some first line holding words i..j, so the problem has optimal substructure: after fixing the first line, the rest must itself be optimal.

Recurrence. dp[n] = 0; dp[i] = min over every j with length(i..j) ≤ W of [$(W - \text{length}(i..j))^2$ + dp[j+1]] when j < n−1, and **0** when j = n−1 (the final line is free). A second array next[i] stores the j+1 that gave the minimum, so the lines are rebuilt by following next[0], next[next[0]], …

Complexity. O(n·k) time where k is the maximum number of words that fit on one line ($O(n^2)$ worst case); O(n) memory.

Source code - task1_wordwrap.py

```
"""Task 1 - Word wrap with minimum total penalty (Dynamic Programming).
Penalty of a NON-final line = (unused spaces)^2 ; final line penalty = 0.
dp[i] = minimum penalty to lay out words[i..n-1]   (suffix formulation)
dp[n] = 0 (no words left). Answer = dp[0].
"""
INF = float("inf")

def validate(words, width):
    if width <= 0:
        raise ValueError("Line width must be positive")
    for w in words:
        if len(w) > width:
            raise ValueError(f"Word '{w}' (len {len(w)}) is longer than width {width}")

def line_length(words, i, j):
    """Length of words[i..j] (inclusive) separated by single spaces."""
    return sum(len(w) for w in words[i:j + 1]) + (j - i)

def solve_word_wrap(words, width, trace=None):
    validate(words, width)
    n = len(words)
    if n == 0:
        return 0, []
    dp = [INF] * (n + 1)
    nxt = [-1] * (n + 1)          # nxt[i] = index of first word of the NEXT line
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        for j in range(i, n):      # line holds words i..j
            length = line_length(words, i, j)
            if length > width:
```

```
            break
        spare = width - length
        if j == n - 1:
            cost = 0                    # final line: free
        else:
            cost = spare ** 2 + dp[j + 1]
        if trace is not None:
            trace.append((i, j, length, spare, cost))
        if cost < dp[i]:
            dp[i], nxt[i] = cost, j + 1
    # note: for j==n-1, cost already excludes dp[n]=0 so equal
  lines, i = [], 0
  while i < n:
    j = nxt[i]
    lines.append((i, j - 1))
    i = j
  return dp, lines

def report(words, width, show_states=False):
  trace = []
  dp, lines = solve_word_wrap(words, width, trace)
  n = len(words)
  print(f"\n=== W = {width} ===")
  if show_states:
    print("DP table: dp[i] = min penalty for words i..n-1")
    print(" i | word   | dp[i]")
    for i in range(n + 1):
      print(f"{i:2d} | {(words[i] if i<n else '<end>'):6s} | {dp[i]}")
    print("\nCandidate evaluation (line = words i..j):")
    print(" i  j  len spare  cost(=spare^2+dp[j+1], final=0)")
    for (i, j, L, s, c) in trace:
      print(f"{i:2d} {j:2d} {L:4d} {s:5d}  {c}")
  print(f"\nMinimum total penalty = {dp[0]}")
  print("Line | Words                          | Len | Unused | Penalty")
  total = 0
  for k, (i, j) in enumerate(lines):
    text = " ".join(words[i:j + 1])
    spare = width - len(text)
    pen = 0 if j == n - 1 else spare ** 2
    total += pen
    tag = " (final)" if j == n - 1 else ""
    print(f"{k+1:4d} | {text:30s} | {len(text):3d} | {spare:6d} | {pen}{tag}")
  assert total == dp[0], "reconstruction mismatch"
  return dp[0], lines

if __name__ == "__main__":
  text = "the quick brown fox jumps over the lazy dog near the river bank today"
  words = text.split()
  print("Words:", list(enumerate(words)))
  report(words, 20, show_states=True)
  print("\n\n##### REQUIREMENT CHANGE RUNS #####")
  for W in (14, 25, 32):
    report(words, W)
  print("\n\n##### BOUNDARY TESTS #####")
  # B1: every word exactly fills the line
  print("\n-- B1: words exactly equal to W, W=5 --")
  report(["hello", "world", "again"], 5)
  # B2: a word longer than W
  print("\n-- B2: word longer than W --")
  try:
    report(["extraordinary", "cat"], 5)
  except ValueError as e:
    print("Rejected correctly:", e)
```

```
# B3: single word
print("\n-- B3: single word --")
report(["hi"], 10)
# B4: everything fits on one line (final line free -> penalty 0)
print("\n-- B4: whole text fits in one line --")
report(["a", "b", "c"], 50)
# B5: greedy vs DP difference (shows DP != greedy)
print("\n-- B5: classic greedy-vs-DP case, W=6 --")
report(["aaa", "bb", "cc", "ddddd"], 6)
# B6: empty
print("\n-- B6: empty input --")
print(solve_word_wrap([], 10))
```

Test input (14 words, W = 20): "the quick brown fox jumps over the lazy dog near the river bank today"

Line	Words on the line	Length	Unused spaces s	Penalty s^2
1	the quick brown fox	19	1	1
2	jumps over the lazy	19	1	1
3	dog near the river	18	2	4
4	bank today	10	10	0 (final line)
	Minimum total penalty			**6**

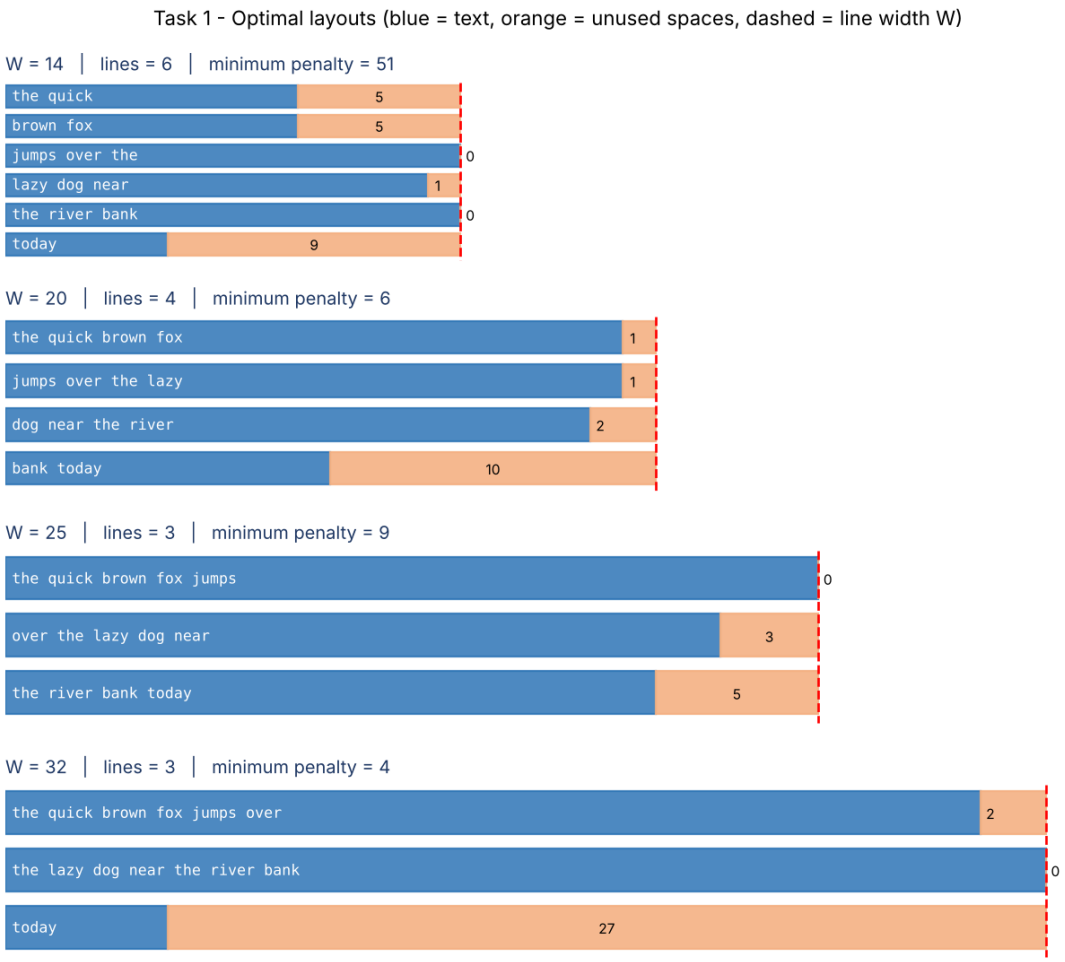


Figure 1.1 - Optimal layouts for W = 14, 20, 25, 32 (orange = unused spaces; red dashed line = W)

1.2 Exposing the DP behaviour

The program prints (a) the dp[] table and (b) every candidate first line it evaluated, with its length, unused spaces and total cost. Table 1.2a is the dp[] array for W = 20.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
word	the	quick	brown	fox	jumps	over	the	lazy	dog	near	the	river	bank	today	(end)
dp[i]	6	30	9	13	5	10	9	9	4	1	0	0	0	0	0

What a displayed state means. dp[i] = the least penalty achievable for the words from index i onward, assuming a fresh line starts at word i. For example dp[4] = 5 means: for "jumps over the lazy dog … today" the best possible total is 5.

How dp[0] = 6 was constructed (candidate table for i = 0; cost = spare² + dp[j+1]):

First line = words 0..j	Length	Unused s	s²	dp[j+1]	Total	Chosen?
0..0 "the"	3	17	289	30	319	
0..1 "the quick"	9	11	121	9	130	
0..2 "the quick brown"	15	5	25	13	38	
0..3 "the quick brown fox"	19	1	1	5	6	**YES (min)**

One word on line 1 costs 17^2 + dp[1] = 289 + 30 = 319; two words cost 121 + 9 = 130; three words 25 + 13 = 38; four words only 1 + 5 = **6**. The 4-word line leaves just 1 unused space and hands a cheap suffix (dp[4] = 5) to the rest. Following the stored choices gives the chain dp[0]=1+dp[4], dp[4]=1+dp[8], dp[8]=4+dp[12], dp[12]=0 → **1 + 1 + 4 + 0 = 6**, exactly the sum of the line penalties in the table above (an assertion in the code checks this).

1.3 Requirement change - three more line widths

Predictions were written from first principles before each run (a wider line should let more words fit, so line 1 changes first; fewer lines usually means fewer penalty terms; a narrower line forces more breaks):

W	Prediction (before running)	Actual result	Match?
14 (narrower)	Arrangement changes. First change on **line 1** (fewer words fit). More lines. Penalty **increases**.	6 lines; line 1 = "the quick" (2 words instead of 4). Penalty **51** (was 6).	Yes
25 (wider)	Arrangement changes. First change on **line 1** (more words fit). Fewer lines. Penalty expected to **decrease**.	3 lines; line 1 = "the quick brown fox jumps" (5 words). Penalty **9** (was 6).	Lines/first change: Yes. Penalty: **No** (increased)
32 (wider)	Arrangement changes. First change on **line 1**. Fewer lines. Penalty **decreases**.	3 lines; line 1 holds 6 words. Penalty **4**.	Yes

Interpretation. Two of three predictions were fully right. The W = 25 penalty prediction was wrong, which is the useful finding: the penalty is **not monotonic in W**. At W = 20 the text happens to pack almost perfectly (unused spaces 1, 1, 2 → penalty 6), whereas at W = 25 the best achievable layout must leave 3 spaces on line 2 (penalty 9). Fewer lines reduce the number of penalty terms but not the size of each term. The structure that DOES always change first is line 1, because the first break point depends directly on W.

Line	Words (W = 14)	Len	Unused	Penalty
1	the quick	9	5	25
2	brown fox	9	5	25
3	jumps over the	14	0	0
4	lazy dog near	13	1	1
5	the river bank	14	0	0
6	today	5	9	0 (final)
	Total			**51**

Line	Words (W = 25)	Len	Unused	Penalty
1	the quick brown fox jumps	25	0	0
2	over the lazy dog near	22	3	9

Line	Words (W = 25)	Len	Unused	Penalty
3	the river bank today	20	5	0 (final)
	Total			**9**

Line	Words (W = 32)	Len	Unused	Penalty
1	the quick brown fox jumps over	30	2	4
2	the lazy dog near the river bank	32	0	0
3	today	5	27	0 (final)
	Total			**4**

1.4 Boundary test

Selected case: words whose length equals W exactly ("hello world again", W = 5). **Why:** the fit test is the most likely place for an off-by-one defect. If the code used $\text{length} < W$ instead of $\text{length} \leq W$, every word would be rejected (or an infinite loop / missing line would appear); if the spare-space calculation were wrong, penalty would be non-zero. Expected: three lines, 0 unused spaces, penalty 0.

Actual output

```
-- B1: words exactly equal to W, W=5 --

=== W = 5 ===

Minimum total penalty = 0
Line | Words                    | Len | Unused | Penalty
   1 | hello                    |   5 |      0 | 0
   2 | world                    |   5 |      0 | 0
   3 | again                    |   5 |      0 | 0 (final)
```

Extra boundary test	Input	Expected	Actual	Defect it would expose
Word longer than W	["extraordinary", "cat"], W = 5	Rejected with an error	ValueError: word longer than width	Infinite loop / silent overflow
Single word	["hi"], W = 10	Penalty 0 (final line)	0	Charging a penalty on the final line (would give 64)
Everything fits on one line	["a","b","c"], W = 50	Penalty 0	0	Same final-line defect
Empty input	[]	Penalty 0, no lines	(0, [])	Index error on empty list

Extra boundary test	Input	Expected	Actual	Defect it would expose
DP vs greedy	["aaa","bb","cc","ddddd"], W = 6	DP < greedy (16)	DP = **10** (aaa / bb cc / ddddd)	A greedy fill gives 'aaa bb' / 'cc' / 'ddddd' = 16

TASK 2 - Greedy vs Dynamic Programming: Build It, Break It, Explain It

Implementation overview

- **Solution A - Fractional Knapsack (greedy):** sort by value/weight ratio, take whole items while they fit, then a fraction of the next item. Reports each item and fraction taken.
- **Solution B - 0/1 Knapsack (DP):** dp[i][c] = best value using the first i items with capacity c; dp[i][c] = max(dp[i−1][c], dp[i−1][c−w_i] + v_i). Items are recovered by backtracking through the table.
- **Extra - greedy intuition applied to 0/1** (take an item whole only if it still fits) - this is what is "broken" in 2.2.

Source code - task2_knapsack.py

```
"""Task 2 - Greedy (Fractional Knapsack) vs Dynamic Programming (0/1 Knapsack)."""
import random, time, functools

class Counter:
    def __init__(self): self.ops = {}
    def add(self, k, n=1): self.ops[k] = self.ops.get(k, 0) + n

# ---------- Solution A : Fractional knapsack (greedy) ----------
def fractional_greedy(items, capacity, cnt=None, verbose=False):
    """items: list of (id, weight, value). Returns (value, [(id, fraction)])."""
    cnt = cnt or Counter()
    ratios = []
    for it in items:
        cnt.add("ratio computations")
        ratios.append((it[2] / it[1], it))
    def cmp(a, b):
        cnt.add("sort comparisons")
        return (b[0] > a[0]) - (b[0] < a[0])
    ratios.sort(key=functools.cmp_to_key(cmp))
    remaining, total, chosen = capacity, 0.0, []
    for r, (iid, w, v) in ratios:
        if remaining <= 0: break
        cnt.add("selection steps")
        if w <= remaining:
            frac = 1.0
        else:
            frac = remaining / w
        remaining -= frac * w
        total += frac * v
        chosen.append((iid, frac))
        if verbose:
            print(f"  take {iid}: ratio={r:.2f}, fraction={frac:.2f}, remaining cap={remaining:.2f}, value so far={total:.2f}")
    return total, chosen

# ---------- Greedy applied to 0/1 (take whole item only if it fits) ----------
```

```
def greedy_01(items, capacity, verbose=False):
    order = sorted(items, key=lambda t: t[2] / t[1], reverse=True)
    remaining, total, chosen, first_commit = capacity, 0, [], None
    for iid, w, v in order:
        if w <= remaining:
            remaining -= w; total += v; chosen.append(iid)
            if first_commit is None: first_commit = iid
            if verbose: print(f"  commit {iid} (ratio {v/w:.2f}) -> remaining cap {remaining}, value
{total}")
        elif verbose:
            print(f"  skip   {iid} (w={w} > remaining {remaining})")
    return total, chosen, first_commit

# ---------------- Solution B : 0/1 knapsack (DP) ----------------
def knapsack_dp(items, capacity, cnt=None):
    """dp[i][c] = best value using first i items with capacity c."""
    cnt = cnt or Counter()
    n = len(items)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        _, w, v = items[i - 1]
        for c in range(capacity + 1):
            cnt.add("DP cell evaluations")
            dp[i][c] = dp[i - 1][c]
            if w <= c and dp[i - 1][c - w] + v > dp[i][c]:
                dp[i][c] = dp[i - 1][c - w] + v
    chosen, c = [], capacity
    for i in range(n, 0, -1):               # backtrack
        if dp[i][c] != dp[i - 1][c]:
            chosen.append(items[i - 1][0]); c -= items[i - 1][1]
    return dp[n][capacity], chosen[::-1], dp

def show(items):
    print("  ID  Weight  Value  Ratio")
    for i, w, v in items: print(f"  {i:<3} {w:>5}  {v:>5}  {v/w:5.2f}")

if __name__ == "__main__":
    # ---- 2.1 normal case ----
    D1 = [("I1", 10, 60), ("I2", 20, 100), ("I3", 20, 90)]
    print("=== 2.1 NORMAL CASE  (capacity 50) ==="); show(D1)
    print("Greedy (fractional) decision sequence:")
    gv, gch = fractional_greedy(D1, 50, verbose=True)
    print("  Greedy value =", gv, " selected =", gch)
    dv, dch, _ = knapsack_dp(D1, 50)
    print("  DP 0/1 value =", dv, " selected =", dch)

    # ---- 2.2 break the greedy ----
    D2 = [("A", 6, 30), ("B", 5, 24), ("C", 5, 24)]
    print("\n=== 2.2 BREAKING GREEDY  (capacity 10) ==="); show(D2)
    print("Greedy intuition applied to 0/1:")
    g01, gsel, first = greedy_01(D2, 10, verbose=True)
    dv, dsel, table = knapsack_dp(D2, 10)
    print(f"  Greedy 0/1 value = {g01}, items = {gsel}")
    print(f"  DP optimum       = {dv}, items = {dsel}")
    print(f"  Greedy != DP ? {g01 != dv}   (gap = {dv-g01})")
    print(f"  First harmful commitment: item {first}")
    fv, fch = fractional_greedy(D2, 10)
    print(f"  (Fractional relaxation = {fv:.1f}  -> upper bound, not achievable in 0/1)")
    print("  DP table (rows = items considered, cols = capacity 0..10):")
    for i, row in enumerate(table): print("   ", i, row)

    # ---- 2.3 engineering comparison ----
    print("\n=== 2.3 ENGINEERING COMPARISON ===")
```

```
    print(" n  | Cap |  Greedy ops (ratio+sort cmp+select) | DP cell evals | Greedy memory (items) | DP memory (cells) | Greedy val | DP val | Quality")
    random.seed(7)
    for n, cap in [(5, 20), (10, 50), (20, 100), (50, 200), (100, 500), (200, 1000)]:
        items = [(f"X{i}", random.randint(1, 30), random.randint(1, 100)) for i in range(n)]
        c1, c2 = Counter(), Counter()
        _, chosen = fractional_greedy(items, cap, c1)
        gops = sum(c1.ops.values())
        dv,  ,   = knapsack_dp(items, cap, c2)
        g01, _, _ = greedy_01(items, cap)
        dpops = c2.ops["DP cell evaluations"]
        print(f"{n:3d} | {cap:4d} | {gops:8d} | {dpops:10d} | {n:6d} | {(n+1)*(cap+1):8d} | {g01:5d} | {dv:5d} | {100*g01/dv:5.1f}%")

    print("\nQuality on 500 random instances (n=10, cap=40, w in 1..20, v in 1..50):")
    random.seed(2024)
    ratios, equal = [], 0
    for   in range(500):
        items = [(f"X{i}", random.randint(1, 20), random.randint(1, 50)) for i in range(10)]
        g01, _, _ = greedy_01(items, 40)
        dv, _, _ = knapsack_dp(items, 40)
        ratios.append(g01 / dv); equal += (g01 == dv)
    print(f"  greedy optimal in {equal}/500 cases; mean quality {100*sum(ratios)/500:.1f}%; worst {100*min(ratios):.1f}%")

    # near-optimal variant: max(greedy, best single item)
    random.seed(2024)
    imp = []
    for _ in range(500):
        items = [(f"X{i}", random.randint(1, 20), random.randint(1, 50)) for i in range(10)]
        g01,  ,   = greedy_01(items, 40)
        best_single = max((v for _, w, v in items if w <= 40), default=0)
        dv, _, _ = knapsack_dp(items, 40)
        imp.append(max(g01, best_single) / dv)
    print(f"  improved greedy (max(greedy, best single item)): mean {100*sum(imp)/500:.1f}%, worst {100*min(imp):.1f}%")

    # worst-case family
    print("\nWorst-case family: item P (w=1,v=2), item Q (w=C, v=C), capacity C")
    for C in (10, 100, 1000):
        items = [("P", 1, 2), ("Q", C, C)]
        g01, _, _ = greedy_01(items, C); dv, _, _ = knapsack_dp(items, C)
        print(f"  C={C:5d}: greedy={g01}, optimum={dv}, quality={100*g01/dv:.1f}%")

    print("\nDP scaling with capacity (n=100): cells = n*(C+1)")
    for C in (100, 10_000, 1_000_000, 100_000_000):
        print(f"  C={C:>11,}: DP cells = {100*(C+1):>15,}  vs greedy ops ~ n log n = {int(100*6.64)+200}")

    print("\nTiming sanity (n=200, C=5000):")
    random.seed(1)
    items = [(f"X{i}", random.randint(1, 50), random.randint(1, 100)) for i in range(200)]
    t = time.perf_counter(); greedy_01(items, 5000); tg = time.perf_counter() - t
    t = time.perf_counter(); knapsack_dp(items, 5000); td = time.perf_counter() - t
    print(f"  greedy {tg*1000:.3f} ms   DP {td*1000:.1f} ms   (ops/memory counts above are the primary evidence)")
```

2.1 Normal case

Capacity = 50. Items sorted by ratio: I1 (10, 60 → 6.0), I2 (20, 100 → 5.0), I3 (20, 90 → 4.5).

Step	Item chosen (highest remaining ratio)	Fraction	Remaining capacity	Value so far
1	I1 (ratio 6.00)	1.00	40	60
2	I2 (ratio 5.00)	1.00	20	160
3	I3 (ratio 4.50)	1.00	0	250

Greedy = 250, DP (0/1) = 250 with items {I1, I2, I3}. Here the greedy choice is optimal because the items fill the knapsack exactly, so no capacity is wasted - and greedy is always optimal for the **fractional** formulation (exchange argument: swapping any weight to a lower-ratio item can only reduce value).

2.2 Breaking the greedy intuition (student-designed counterexample)

Capacity = 10. A = (w 6, v 30, ratio 5.00), B = (w 5, v 24, ratio 4.80), C = (w 5, v 24, ratio 4.80).

Program output

```
Greedy intuition applied to 0/1:
  commit A (ratio 5.00) -> remaining cap 4, value 30
  skip   B (w=5 > remaining 4)
  skip   C (w=5 > remaining 4)
  Greedy 0/1 value = 30, items = ['A']
  DP optimum       = 48, items = ['B', 'C']
  Greedy != DP ? True   (gap = 18)
  First harmful commitment: item A
  (Fractional relaxation = 49.2  -> upper bound, not achievable in 0/1)
```

Method	Items	Value
Greedy ratio rule on 0/1	A	30
DP optimum	B + C	**48**
Fractional relaxation (upper bound)	A + 0.8 B	49.2

Greedy result (30) ≠ DP optimum (48). The first harmful commitment is **taking item A** (highest ratio): it leaves only 4 units of capacity, so neither B nor C (weight 5) can ever fit, and 4 units are wasted. The ratio rule is blind to the 'indivisible' constraint: weight that cannot be used has zero value.

DP table (rows = items considered, columns = capacity 0–10) shows why the optimum appears only at the last row and capacity 10:

Items considered	c=0	c=1	c=2	c=3	c=4	c=5	c=6	c=7	c=8	c=9	c=10
0	0	0	0	0	0	0	0	0	0	0	0
{A}	0	0	0	0	0	0	30	30	30	30	30
{A,B}	0	0	0	0	0	24	30	30	30	30	30
{A,B,C}	0	0	0	0	0	24	30	30	30	30	**48**

2.3 Engineering comparison

Operations were counted inside the code (ratio computations + sort comparisons + selection steps for greedy; one count per DP cell evaluated). Instances are random with seed 7 (reproducible); 'Greedy value' is the 0/1 version of the greedy rule, compared with the DP optimum.

n	C	Greedy ops	DP cell evals	Greedy memory (entries)	DP memory (entries)	Greedy value	DP value	Quality
5	20	18	105	5	126	141	141	100%
10	50	38	510	10	561	294	294	100%
20	100	98	2,020	20	2,121	768	768	100%
50	200	297	10,050	50	10,251	1,472	1,480	99.5%
100	500	689	50,100	100	50,601	3,397	3,409	99.6%
200	1000	1,577	200,200	200	201,201	6,550	6,563	99.8%

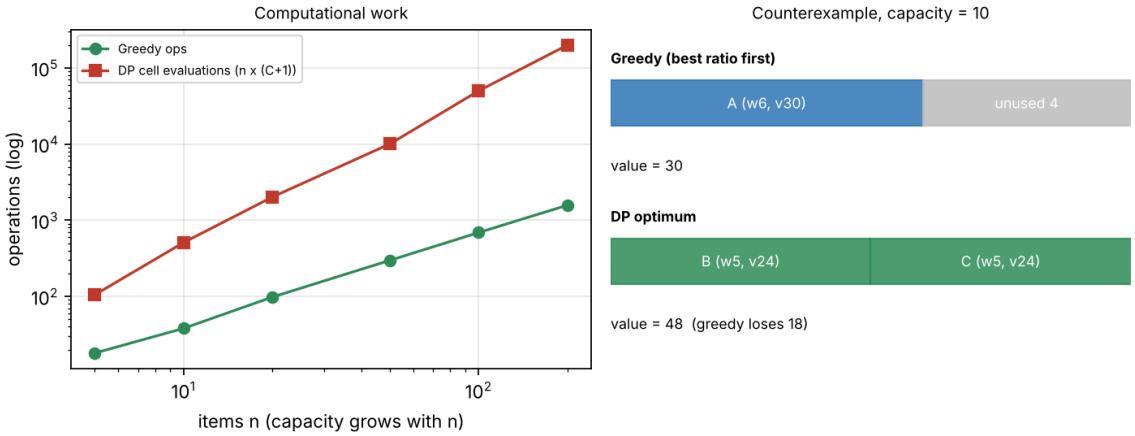


Figure 2.1 - Work grows far faster for DP; counterexample layout

Criterion	Greedy (A / ratio rule)	DP (B)
Computational work	O(n log n): sort + one pass (1,577 ops at n = 200)	O(n·C): 200,200 cell evaluations at n = 200, C = 1000 (127× more)
Memory	O(n) - just the sorted list	O(n·C) table (201,201 entries); O(C) possible if only the value is needed, but backtracking needs the table
Solution quality	Fractional: optimal. 0/1: optimal in 326 / 500 random trials, mean 98.2 %, worst 78.7 %; worst-case family drops to 0.2 %	Always optimal for 0/1
Optimality guarantee	Proven only for fractional; **no guarantee** for 0/1	Proven (exhaustive over subproblems)
Sensitivity to capacity	None (C is not iterated)	Pseudo-polynomial: C = 10^8 → 10^{10} cells for n = 100

Quality worst-case family (item P: w 1, v 2; item Q: w C, v C; capacity C): greedy picks P and gets 2, DP gets C - i.e. 20 %, 2 % and 0.2 % of the optimum for C = 10, 100, 1000. Timing for n = 200,

C = 5000 was 0.05 ms (greedy) vs 217.7 ms (DP) - recorded only as a sanity check, the counts above are the primary evidence.

2.4 Changed requirement - "decision must be extremely fast; near-optimal is acceptable"

Conclusion from the measurements: choose the **greedy strategy** (with a safeguard), keep DP as the offline/verification tool.

- **Speed evidence:** greedy performed 1,577 operations where DP performed 200,200 on the same instance, and the DP count scales with the capacity value, not just the number of items (10^{10} cells at $C = 10^8$, $n = 100$, against 689 greedy operations for the same n).
- **Quality evidence:** on 500 random instances greedy reached the optimum 65 % of the time and averaged 98.2 % of optimal - well inside 'near-optimal'. That is exactly the situation management describes.
- **Risk evidence:** the P/Q family shows greedy can be arbitrarily bad, so the 'acceptable' wording needs a guard. Taking max(greedy result, most valuable single item that fits) fixes the P/Q family (it returns C) and carries the standard ½-approximation guarantee. On the 500 random trials it gave the same average, so it costs nothing in typical cases.
- **When DP still wins:** small capacity values (e.g. $C \leq 10^4$ and $n \leq$ few hundred) where the ~10^6 cells are cheap and exactness has monetary value.

TASK 3 - Routing Engine Under Changing Network Conditions

Graph definition

Directed weighted network with **8 vertices (S, A, B, C, D, E, F, G) and 13 edges**:

Edge	Weight	Edge	Weight	Edge	Weight
S → A	2	A → D	8	D → G	9
S → B	5	B → E	6	E → G	5
A → C	4	D → F	2	C → E	7
B → C	1	E → F	1		
C → D	3	F → G	4		

Source code - task3_routing.py (Dijkstra, Bellman-Ford, Floyd, Warshall)

```
"""Task 3 - Routing engine: Dijkstra, Bellman-Ford, Floyd, Warshall."""
import heapq
INF = float("inf")
VERTS = ["S", "A", "B", "C", "D", "E", "F", "G"]
NORMAL_EDGES = [("S","A",2),("S","B",5),("A","C",4),("B","C",1),("C","D",3),("A","D",8),
                ("B","E",6),("D","F",2),("E","F",1),("F","G",4),("D","G",9),("E","G",5),("C","E",7)]
NEG_EDGE = ("E","D",-5)
CYCLE_EDGE = ("G","E",-7)          # closes E->F->G->E with weight 1+4-7 = -2

def build_adj(edges):
    adj = {v: [] for v in VERTS}
    for u, v, w in edges: adj[u].append((v, w))
    return adj

def path_to(pred, src, t):
```

```
    if t != src and pred.get(t) is None: return None
    p = [t]
    while p[-1] != src: p.append(pred[p[-1]])
    return p[::-1]

# ---------- Stage A : Dijkstra ----------
def dijkstra(edges, src, verbose=False):
    if any(w < 0 for _, _, w in edges):
        print("  [WARNING] negative edge present: Dijkstra's correctness proof no longer applies")
    adj = build_adj(edges)
    dist = {v: INF for v in VERTS}; pred = {v: None for v in VERTS}
    dist[src] = 0; heap = [(0, src)]; final = []
    done = set()
    while heap:
        d, u = heapq.heappop(heap)
        if u in done: continue
        done.add(u); final.append(u)              # u is FINALISED: never revisited
        if verbose: print(f"  finalise {u} (dist={d})")
        for v, w in adj[u]:
            if v not in done and d + w < dist[v]:
                dist[v] = d + w; pred[v] = u
                heapq.heappush(heap, (dist[v], v))
                if verbose: print(f"      relax {u}->{v} (w={w}): dist[{v}] = {dist[v]}")
    return dist, pred, final

# ---------- Stage B/C : Bellman-Ford ----------
def bellman_ford(edges, src, verbose=False):
    dist = {v: INF for v in VERTS}; pred = {v: None for v in VERTS}
    dist[src] = 0
    V = len(VERTS)
    for it in range(1, V):
        changed = False
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w; pred[v] = u; changed = True
        if verbose: print(f"  pass {it}: " + ", ".join(f"{x}={dist[x]}" for x in VERTS))
        if not changed:
            if verbose: print(f"  no change in pass {it} -> converged early")
            break
    # V-th pass: any further relaxation proves a reachable negative cycle
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            # walk back V steps to land INSIDE the cycle, then collect it
            x = v
            for _ in range(V): x = pred[x] if pred[x] is not None else x
            cyc, cur = [x], pred[x]
            while cur != x: cyc.append(cur); cur = pred[cur]
            cyc.append(x); cyc.reverse()
            return dist, pred, cyc
    return dist, pred, None

# ---------- Stage D : Floyd & Warshall ----------
def floyd_warshall(edges):
    n = len(VERTS); idx = {v: i for i, v in enumerate(VERTS)}
    d = [[INF]*n for _ in range(n)]
    for i in range(n): d[i][i] = 0
    for u, v, w in edges: d[idx[u]][idx[v]] = min(d[idx[u]][idx[v]], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]: d[i][j] = d[i][k] + d[k][j]
    neg_cycle = any(d[i][i] < 0 for i in range(n))
    return d, neg_cycle
```

```
def warshall(edges):
    n = len(VERTS); idx = {v: i for i, v in enumerate(VERTS)}
    r = [[0]*n for _ in range(n)]
    for i in range(n): r[i][i] = 1
    for u, v, _ in edges: r[idx[u]][idx[v]] = 1
    for k in range(n):
        for i in range(n):
            for j in range(n):
                r[i][j] = r[i][j] or (r[i][k] and r[k][j])
    return r

def print_dist(src, dist, pred):
    print("  Vertex | Dist | Pred | Path")
    for v in VERTS:
        p = path_to(pred, src, v)
        print(f"  {v:6s} | {str(dist[v]):4s} | {str(pred[v]):4s} | {' -> '.join(p) if p else 'unreachable'}")

def print_matrix(m, fmt=lambda x: "INF" if x == INF else str(x)):
    print("       " + " ".join(f"{v:>4s}" for v in VERTS))
    for i, v in enumerate(VERTS):
        print(f"  {v:>4s} " + " ".join(f"{fmt(x):>4s}" for x in m[i]))

if __name__ == "__main__":
    print(f"Graph: {len(VERTS)} vertices, {len(NORMAL_EDGES)} directed edges")
    print("Edges:", NORMAL_EDGES)
    print("\n===== STAGE A: Dijkstra, source S =====")
    dist, pred, order = dijkstra(NORMAL_EDGES, "S", verbose=True)
    print_dist("S", dist, pred)
    print("  Finalisation order:", " -> ".join(order))

    print("\n===== STAGE B: add negative edge E->D (-5) =====")
    edges_b = NORMAL_EDGES + [NEG_EDGE]
    print("Dijkstra on modified graph:")
    d2, p2, o2 = dijkstra(edges_b, "S", verbose=True)
    print_dist("S", d2, p2); print("  Finalisation order:", " -> ".join(o2))
    print("\nBellman-Ford on modified graph:")
    d3, p3, cyc = bellman_ford(edges_b, "S", verbose=True)
    print_dist("S", d3, p3); print("  Negative cycle:", cyc)
    print("\nComparison (Dijkstra vs Bellman-Ford):")
    print("  Vertex | Dijkstra | Bellman-Ford | Match?")
    for v in VERTS:
        print(f"  {v:6s} | {d2[v]:8} | {d3[v]:12} | {'yes' if d2[v]==d3[v] else 'NO  <-- Dijkstra
wrong'}")

    print("\n===== STAGE C: add G->E (-7): reachable negative cycle E->F->G->E =====")
    edges_c = edges_b + [CYCLE_EDGE]
    d4, p4, cyc = bellman_ford(edges_c, "S", verbose=True)
    if cyc:
        w = sum(next(w for a, b, w in edges_c if a == x and b == y) for x, y in zip(cyc, cyc[1:]))
        print("  NEGATIVE CYCLE DETECTED:", " -> ".join(cyc), f"(total weight {w})")
        print("  Shortest paths are undefined for vertices reachable from the cycle; results withheld.")
    # boundary: unreachable negative cycle should NOT raise a false alarm
    print("\n  Control test: negative cycle NOT reachable from source -> must not be reported")
    VERTS_BACKUP = list(VERTS)
    VERTS.extend(["X", "Y"])
    edges_ctl = NORMAL_EDGES + [("X","Y",1),("Y","X",-3)]
    _, _, cyc2 = bellman_ford(edges_ctl, "S")
    print("  cycle reported:", cyc2)
    VERTS[:] = VERTS_BACKUP

    print("\n===== STAGE D: Floyd (all-pairs distance) on graph with E->D(-5) =====")
    fm, nc = floyd_warshall(edges_b)
```

```
    print("  negative cycle (diagonal < 0)?", nc)
    print  matrix(fm)
    print("\n  Warshall reachability matrix (1 = reachable):")
    print_matrix(warshall(edges_b), fmt=str)
    fm2, nc2 = floyd_warshall(edges_c)
    print("\n  Floyd on Stage-C graph: negative diagonal entries ->", [VERTS[i] for i in
range(len(VERTS)) if fm2[i][i] < 0], "negative cycle detected =", nc2)
```

Stage A - Normal routing with Dijkstra (source S)

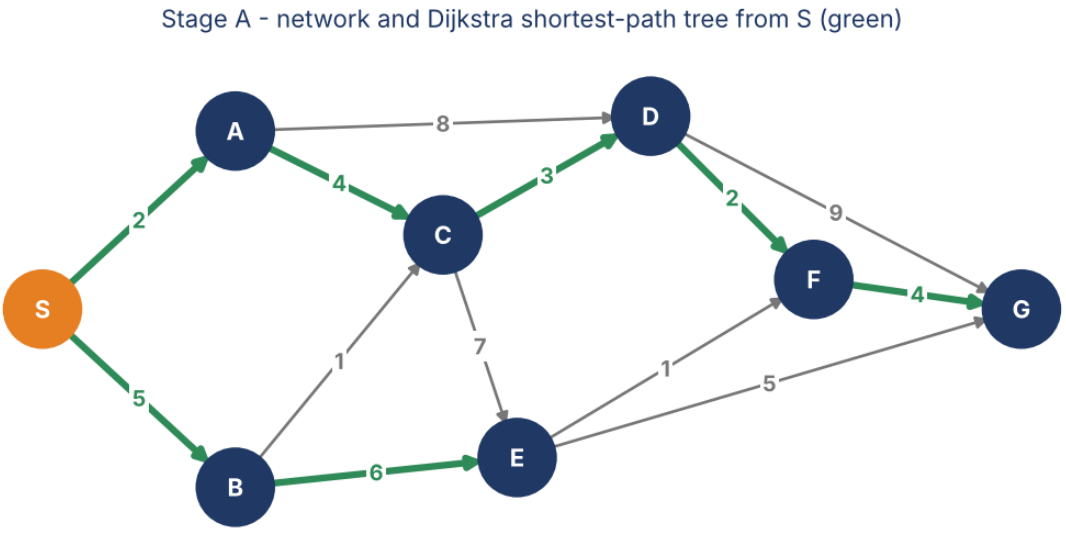


Figure 3.1 - Network and Dijkstra shortest-path tree (green)

Vertex	Shortest distance	Predecessor	Actual path
S	0	-	S
A	2	S	S -> A
B	5	S	S -> B
C	6	A	S -> A -> C
D	9	C	S -> A -> C -> D
E	11	B	S -> B -> E
F	11	D	S -> A -> C -> D -> F
G	15	F	S -> A -> C -> D -> F -> G

Order in which vertices were finalised: S → A → B → C → D → E → F → G

Trace of Dijkstra (finalise / relax events)

```
===== STAGE A: Dijkstra, source S =====
  finalise S (dist=0)
     relax S->A (w=2): dist[A] = 2
     relax S->B (w=5): dist[B] = 5
  finalise A (dist=2)
     relax A->C (w=4): dist[C] = 6
     relax A->D (w=8): dist[D] = 10
  finalise B (dist=5)
     relax B->E (w=6): dist[E] = 11
  finalise C (dist=6)
```

```
   relax C->D (w=3): dist[D] = 9
finalise D (dist=9)
   relax D->F (w=2): dist[F] = 11
   relax D->G (w=9): dist[G] = 18
finalise E (dist=11)
   relax E->G (w=5): dist[G] = 16
finalise F (dist=11)
   relax F->G (w=4): dist[G] = 15
finalise G (dist=15)
```

Stage B - Negative edge E → D (weight −5)

> **Prediction (before running)** Dijkstra will **not** remain valid. D is finalised at 9 (via S→A→C→D) before E is finalised at 11, so Dijkstra will never see that S→B→E→D costs 5 + 6 − 5 = **6**. The route to D, and everything downstream of D (F and G), is likely to be wrong. The edge was chosen deliberately: it starts at a vertex finalised LATER than its target, which is the exact condition that breaks Dijkstra, and it creates no negative cycle (D cannot reach E).

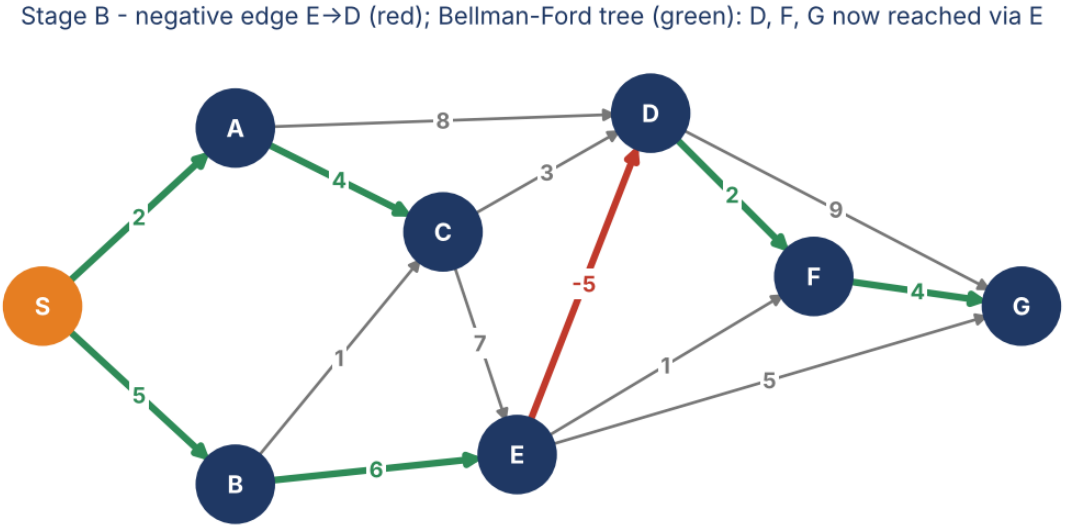


Figure 3.2 - Negative edge (red) and the correct shortest-path tree from Bellman-Ford (green)

Vertex	Dijkstra distance	Dijkstra path	Bellman-Ford distance	Bellman-Ford path	Verdict
S	0	S	0	S	same
A	2	S -> A	2	S -> A	same
B	5	S -> B	5	S -> B	same
C	6	S -> A -> C	6	S -> A -> C	same
D	9	S -> A -> C -> D	6	S -> B -> E -> D	**Dijkstra wrong**
E	11	S -> B -> E	11	S -> B -> E	same
F	11	S -> A -> C -> D -> F	8	S -> B -> E -> D -> F	**Dijkstra wrong**

Vertex	Dijkstra distance	Dijkstra path	Bellman-Ford distance	Bellman-Ford path	Verdict
G	15	S -> A -> C -> D -> F -> G	12	S -> B -> E -> D -> F -> G	**Dijkstra wrong**

Result: prediction confirmed. Dijkstra reports D = 9, F = 11, G = 15; the true values are D = 6, F = 8, G = 12 (each 3 units lower than Dijkstra's value). The decision that changes is the route to D: S→A→C→D (cost 9) is replaced by S→B→E→D (cost 6), and F and G inherit the shorter route.
Why Dijkstra fails: its invariant 'the vertex with the smallest tentative distance is final' relies on all edge weights being ≥ 0, so a later vertex can never improve an earlier one. A negative edge breaks that. (Variants that re-open finalised vertices would repair this graph but lose the guarantee and loop forever on negative cycles - hence the code prints a warning whenever a negative edge is present.)

Bellman-Ford distance after each pass (converged in pass 3; the third pass changes nothing):

Bellman-Ford trace

```
pass 1: S=0, A=2, B=5, C=6, D=6, E=11, F=11, G=15
pass 2: S=0, A=2, B=5, C=6, D=6, E=11, F=8, G=12
pass 3: S=0, A=2, B=5, C=6, D=6, E=11, F=8, G=12
no change in pass 3 -> converged early
```

Stage C - Production failure: reachable negative-weight cycle

Edge G → E (−7) is added. It closes cycles such as D→F→G→E→D with weight 2 + 4 − 7 − 5 = **−6** (and E→F→G→E = −2); both are reachable from S, so no finite shortest path exists for D, E, F, G.

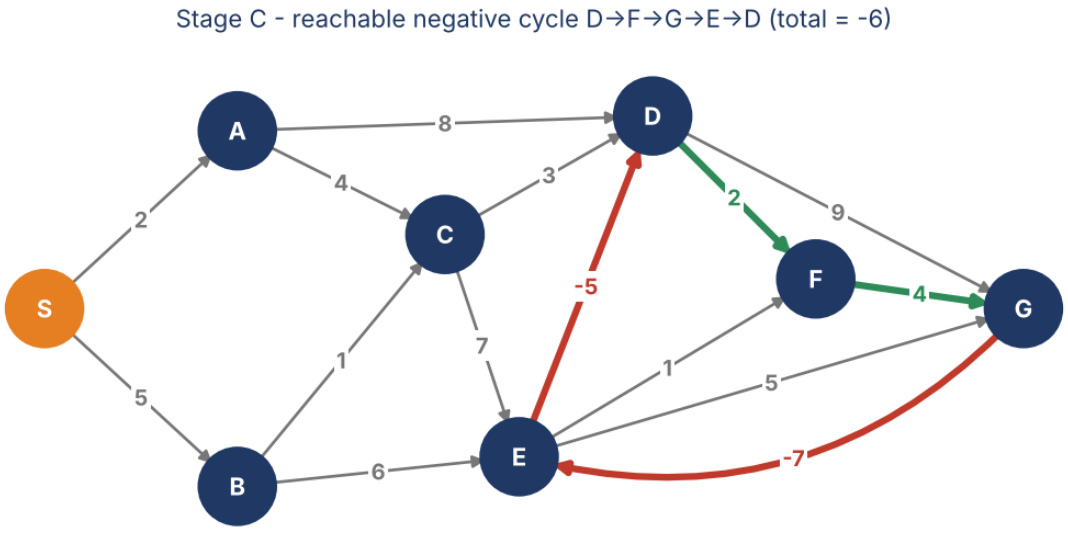


Figure 3.3 - Reachable negative cycle

Program output (Stage C)

```
pass 1: S=0, A=2, B=5, C=6, D=6, E=8, F=11, G=15
pass 2: S=0, A=2, B=5, C=6, D=3, E=5, F=8, G=12
pass 3: S=0, A=2, B=5, C=6, D=0, E=2, F=5, G=9
pass 4: S=0, A=2, B=5, C=6, D=-3, E=-1, F=2, G=6
pass 5: S=0, A=2, B=5, C=6, D=-6, E=-4, F=-1, G=3
pass 6: S=0, A=2, B=5, C=6, D=-9, E=-7, F=-4, G=0
```

```
pass 7: S=0, A=2, B=5, C=6, D=-12, E=-10, F=-7, G=-3
NEGATIVE CYCLE DETECTED: F -> G -> E -> D -> F (total weight -6)
Shortest paths are undefined for vertices reachable from the cycle; results withheld.
```

How detection works. A shortest simple path has at most V−1 = 7 edges, so after V−1 passes of relaxing every edge all distances must be final. The implementation performs one more (the V-th) check; if any edge can still be relaxed, a negative cycle reachable from the source exists. The distances in the trace above keep falling in every pass (D: 6 → 3 → 0 → −3 → … → −12), the symptom of the cycle. To report the cycle, the program follows predecessor pointers V steps back (guaranteeing it is inside the cycle) and then walks the cycle until it repeats. The algorithm raises the error and withholds all distances instead of returning misleading numbers.

Control (false-alarm) test: a negative cycle X↔Y that is NOT reachable from S is added; the program correctly reports no cycle, because relaxations only start from vertices with finite distance.

Stage D - All-pairs cost (Floyd) and reachability (Warshall)

Run on the Stage-B network (negative edge E→D present, no negative cycle). Floyd: d[i][j] = min(d[i][j], d[i][k] + d[k][j]) for k = S..G. Warshall: r[i][j] = r[i][j] OR (r[i][k] AND r[k][j]).

Floyd distance matrix (INF = no path):

From \ To	S	A	B	C	D	E	F	G
S	0	2	5	6	6	11	8	12
A	INF	0	INF	4	6	11	8	12
B	INF	INF	0	1	1	6	3	7
C	INF	INF	INF	0	2	7	4	8
D	INF	INF	INF	INF	0	INF	2	6
E	INF	INF	INF	INF	-5	0	-3	1
F	INF	INF	INF	INF	INF	INF	0	4
G	INF	INF	INF	INF	INF	INF	INF	0

Warshall reachability matrix (1 = reachable):

From \ To	S	A	B	C	D	E	F	G
S	1	1	1	1	1	1	1	1
A	0	1	0	1	1	1	1	1
B	0	0	1	1	1	1	1	1
C	0	0	0	1	1	1	1	1
D	0	0	0	0	1	0	1	1
E	0	0	0	0	1	1	1	1
F	0	0	0	0	0	0	1	1
G	0	0	0	0	0	0	0	1

Interpretation. All diagonal entries are 0, so there is no negative cycle. Row S of the Floyd matrix equals the Bellman-Ford result (A 2, B 5, C 6, D 6, E 11, F 8, G 12), which cross-validates both

algorithms. Negative entries (E→D = −5, E→F = −3) are valid shortest paths. The matrix is not symmetric: E reaches D but D cannot reach E; S reaches every location while no location can reach S; G is a sink (it reaches only itself). Warshall's matrix is exactly the finite pattern of Floyd's matrix, as it must be. Running Floyd on the Stage-C graph produced negative diagonal entries for D, E, F, G, i.e. Floyd also detects the cycle (via d[i][i] < 0) and its matrix must then be rejected.

TASK 4 - Smart Manufacturing Optimisation: Assembly-Line Scheduling

Task 4.1 - Optimisation engine

Model. f_l[j] = minimum time for a product to leave station j on line l. With a_l[j] = processing time, t_l[j] = time to transfer away from line l after station j, e_l = entry time, x_l = exit time:

- f_l[1] = e_l + a_l[1]
- f_l[j] = a_l[j] + min(f_l[j−1] (stay), f_other[j−1] + t_other[j−1] (transfer))
- Answer = min(f_1[n] + x_1, f_2[n] + x_2). A predecessor array source_l[j] records which line the product came from; the route is rebuilt backwards from the best last line.

Input data (6 stations): a_1 = [7, 9, 3, 4, 8, 4], a_2 = [8, 5, 6, 4, 5, 7], t_1 = [2, 3, 1, 3, 4], t_2 = [2, 1, 2, 2, 1], entry e = [2, 4], exit x = [3, 2].

Source code - task4_assembly.py

```
"""Task 4 - Assembly-line scheduling (2 lines) with Dynamic Programming."""
INF = float("inf")

def assembly_line(a, t, e, x, verbose=False):
    """a[l][j]: processing time of station j on line l (l=0,1)
       t[l][j]: transfer time leaving line l AFTER station j (j=0..n-2), to the other line
       e[l]: entry time, x[l]: exit time. Returns (min time, route list, trace rows)."""
    n = len(a[0])
    if n == 0 or len(a[1]) != n: raise ValueError("both lines need the same number (>=1) of stations")
    f = [[0]*n for _ in range(2)]; src = [[None]*n for _ in range(2)]
    trace = []
    f[0][0], f[1][0] = e[0] + a[0][0], e[1] + a[1][0]
    for j in range(1, n):
        row = {"station": j + 1}
        for l in (0, 1):
            o = 1 - l
            stay = f[l][j-1] + a[l][j]
            move = f[o][j-1] + t[o][j-1] + a[l][j]
            if stay <= move: f[l][j], src[l][j] = stay, l
            else:            f[l][j], src[l][j] = move, o
            row[l] = (stay, move, "Stay" if stay <= move else "Transfer")
        trace.append(row)
    fin = [f[0][n-1] + x[0], f[1][n-1] + x[1]]
    last = 0 if fin[0] <= fin[1] else 1
    best = fin[last]
    route, l = [], last
    for j in range(n-1, -1, -1):
        route.append((l, j)); l = src[l][j] if j > 0 else l
    route.reverse()
    return best, route, trace, f, fin

def print_route(route, best, e):
    print("Entry")
    first = True
    prev = None
    for (l, j) in route:
        if prev is not None and prev != l:
```

```
            print(f" |  Transfer (Line {prev+1} -> Line {l+1})")
        print(" v")
        print(f"Line {l+1} - Station {j+1}")
        prev = l
    print(" v\nExit")
    print(f"Minimum Production Time = {best}")
    transfers = [(j+1) for k, (l, j) in enumerate(route[1:], 1) if route[k-1][0] != l]
    print("Transfer points (before station):", transfers if transfers else "none")
    print("Line per station:", [l+1 for l,    in route])

def print_trace(trace, f):
    print("Station | L1:Stay | L1:Transfer | L1 Sel      | L2:Stay | L2:Transfer | L2 Sel")
    print(f"   1    |   -    |    -       | entry+a1={f[0][0]:<3}|   -    |    -       | entry+a1={f[1][0]}")
    for r in trace:
        s1, m1, c1 = r[0]; s2, m2, c2 = r[1]
        print(f"{r['station']:^8}| {s1:^7} | {m1:^11} | {c1:<11} | {s2:^7} | {m2:^11} | {c2}")

# --------- data: 6 stations ----------
A1 = [7, 9, 3, 4, 8, 4]
A2 = [8, 5, 6, 4, 5, 7]
T1 = [2, 3, 1, 3, 4]      # leaving line 1 after station j
T2 = [2, 1, 2, 2, 1]      # leaving line 2 after station j
E = [2, 4]; X = [3, 2]

def run(title, a, t, e=E, x=X, show=True):
    best, route, trace, f, fin = assembly_line(a, t, e, x)
    print(f"\n##### {title} #####")
    if show:
        print_trace(trace, f)
        print(f"f1[n]+x1 = {fin[0]},  f2[n]+x2 = {fin[1]}  -> best = {best}")
    print_route(route, best, e)
    return best, route

if __name__ == "__main__":
    print("Input: a1 =", A1, " a2 =", A2, " t1 =", T1, " t2 =", T2, " e =", E, " x =", X)
    base, r0 = run("BASELINE", [A1, A2], [T1, T2])

    print("\n\n===== 4.3 PREDICT BEFORE EXECUTE: raise t2[2] (leave line 2 after station 2)
from 1 to 20 =====")
    T2b = T2.copy(); T2b[1] = 20
    b3, r3 = run("MODIFIED TRANSFER TIME", [A1, A2], [T1, T2b])
    print("Route changed?", [l for l,_ in r0] != [l for l,_ in r3], " | time:", base, "->", b3)

    print("\n\n===== 4.4 DISRUPTION: station 3 on Line 1 slows from 3 to 15 =====")
    A1d = A1.copy(); A1d[2] = 15
    b4, r4 = run("DISRUPTION (Line1, Station3 = 15)", [A1d, A2], [T1, T2])
    print("Route changed?", [l for l,_ in r0] != [l for l,_ in r4], " | time:", base, "->", b4)

    print("\n\n===== Boundary tests =====")
    b, r, *_ = assembly_line([[5],[3]], [[],[]], [1,1], [1,1])
    print("1 station only: best =", b, " route =", [(l+1, j+1) for l, j in r])
    b, r, *_ = assembly_line([[4,4,4],[4,4,4]], [[0,0],[0,0]], [0,0], [0,0])
    print("symmetric tie: best =", b, " route =", [(l+1, j+1) for l, j in r], "(tie -> stay, no pointless
transfer)")
    try: assembly_line([[1,2],[1]], [[1],[1]], [0,0],[0,0])
    except ValueError as err: print("invalid input rejected:", err)
    # brute-force validation
    import itertools
    def brute(a, t, e, x):
        n = len(a[0]); best = INF
        for combo in itertools.product((0,1), repeat=n):
            c = e[combo[0]] + a[combo[0]][0]
            for j in range(1, n):
```

```
            c += a[combo[j]][j] + (t[combo[j-1]][j-1] if combo[j] != combo[j-1] else 0)
        best = min(best, c + x[combo[-1]])
    return best
for name, aa, tt in [("baseline", [A1,A2], [T1,T2]), ("4.3", [A1,A2], [T1,T2b]), ("4.4", [A1d,A2],
[T1,T2])]:
    print(f"brute force ({name}) =", brute(aa, tt, E, X), "| DP =", assembly_line(aa, tt, E, X)[0])
```

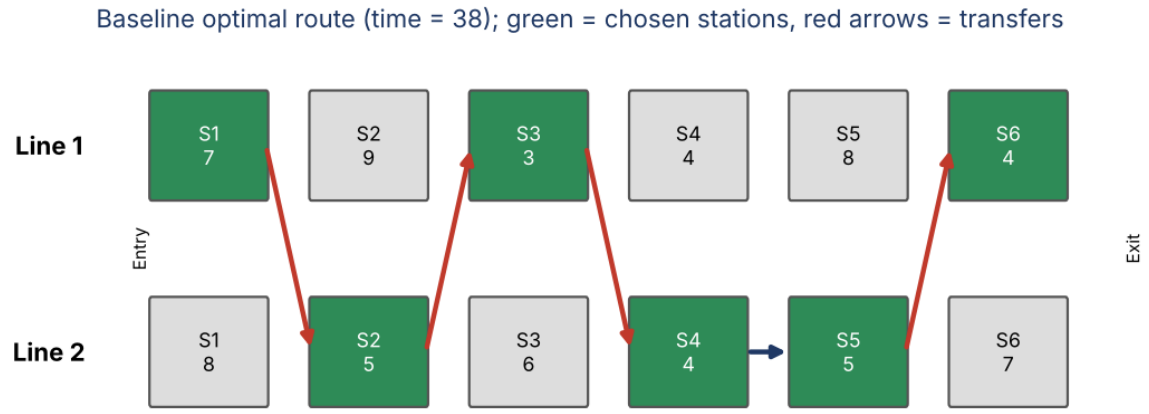


Figure 4.1 - Optimal route (green); transfers are the red arrows

Final optimal route:

```
Entry
 v
Line 1 - Station 1
 |  Transfer (Line 1 -> Line 2)
 v
Line 2 - Station 2
 |  Transfer (Line 2 -> Line 1)
 v
Line 1 - Station 3
 |  Transfer (Line 1 -> Line 2)
 v
Line 2 - Station 4
 v
Line 2 - Station 5
 |  Transfer (Line 2 -> Line 1)
 v
Line 1 - Station 6
 v
Exit
Minimum Production Time = 38
```

Station	1	2	3	4	5	6
Line selected	1	2	1	2	2	1
Transfer before this station?	-	Yes (1→2)	Yes (2→1)	Yes (1→2)	No	Yes (2→1)

Validation: a brute-force enumeration of all $2^6 = 64$ line sequences returned 38, equal to the DP result (also 39 and 39 for the two modified runs below).

Task 4.2 - Decision trace (baseline)

Each cell compares the cost of reaching a line at station j by staying (previous time on the same line + a) versus transferring (previous time on the other line + transfer time + a). Station 1 is just entry time + a. The cheaper option is selected (ties → stay).

Station	Line 1: stay	Line 1: transfer	Line 1 selected	Line 2: stay	Line 2: transfer	Line 2 selected
1	-	-	entry+a = 9	-	-	entry+a = 12
2	18	23	Stay	17	16	Transfer
3	21	20	Transfer	22	27	Stay
4	24	28	Stay	26	25	Transfer
5	32	35	Stay	30	32	Stay
6	36	35	Transfer	37	43	Stay

At the end: $f_1[6] + x_1 = 36 + 3 = 38$ versus $f_2[6] + x_2 = 37 + 2 = 39$, so the product leaves via Line 1 with 38. **Optimal substructure made visible:** every value in a row is built only from the two values of the previous row, e.g. $f_1[3] = \min(18 + 3, 16 + 1 + 3) = 20$ is built only from the two station-2 values ($f_1[2] = 18$ and $f_2[2] = 16$), and the transfer option wins (20 against 21). The optimal route to the last station contains optimal routes to each earlier station on it.

Task 4.3 - Predict before you execute

Change: $t_2[2]$ (transfer from Line 2 to Line 1 after station 2) is raised from 1 to **20**.

> **Prediction (recorded first)** (1) The optimal route **will change**. (2) The decision most likely affected is the **Line 2 → Line 1 transfer before station 3**, which the baseline uses at a cost of only 1. (3) Total production time will **increase** (the cheap transfer is gone), but only slightly, because staying on Line 2 through station 3 is a close alternative (staying on Line 2 costs 22 against 20 at station 3).

Station	Line 1: stay	Line 1: transfer	Line 1 selected	Line 2: stay	Line 2: transfer	Line 2 selected
1	-	-	entry+a = 9	-	-	entry+a = 12
2	18	23	Stay	17	16	Transfer
3	21	39	Stay	22	27	Stay
4	25	28	Stay	26	26	Stay
5	33	36	Stay	31	33	Stay
6	37	36	Transfer	38	44	Stay

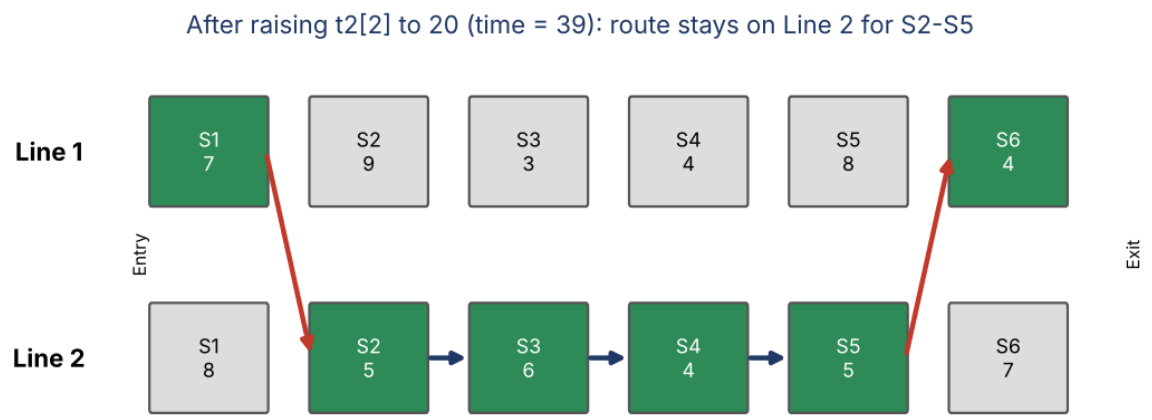


Figure 4.2 - Route after t2[2] = 20

	Baseline	After t2[2] = 20
Line per station	1, 2, 1, 2, 2, 1	1, 2, 2, 2, 2, 1
Transfers (before station)	2, 3, 4, 6	2, 6
Minimum production time	38	39

Comparison: all three predictions were correct. The route changed exactly at the predicted decision: the product now stays on Line 2 for stations 2–5, so the transfers before stations 3 and 4 disappear (the Line 1 detour at station 3 was only worthwhile because the transfer was nearly free). The time rose by only 1 (38 → 39) even though the transfer became 20× more expensive - the DP routed around the expensive edge.

Task 4.4 - Production disruption

Station 3 on Line 1 slows from 3 to **15** (a_1[3] = 15). All other data unchanged.

Station	Line 1: stay	Line 1: transfer	Line 1 selected	Line 2: stay	Line 2: transfer	Line 2 selected
1	-	-	entry+a = 9	-	-	entry+a = 12
2	18	23	Stay	17	16	Transfer
3	33	32	Transfer	22	27	Stay
4	36	28	Transfer	26	37	Stay
5	36	36	Stay	31	36	Stay
6	40	36	Transfer	38	47	Stay

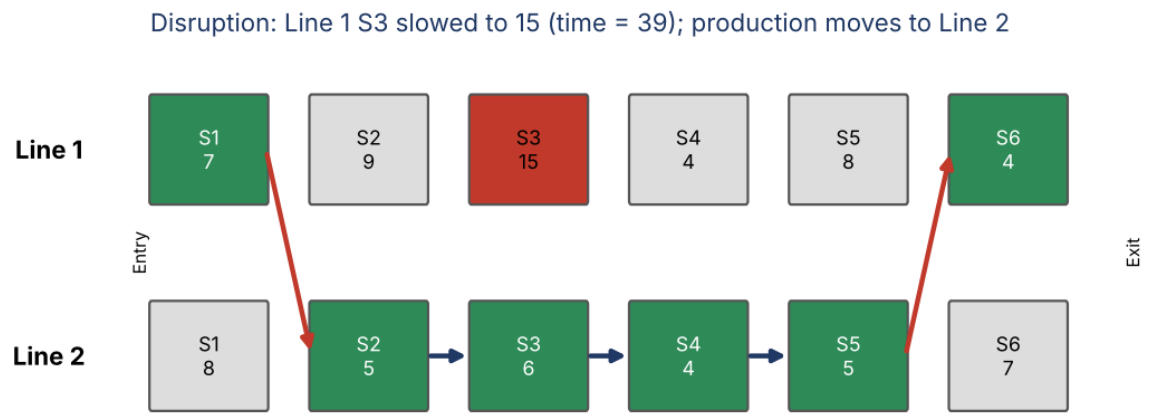


Figure 4.3 - Disruption at Line 1, Station 3 (red box): production moves to Line 2

Question	Answer from the run
Does the optimal route change?	Yes: [1,2,1,2,2,1] → [1,2,2,2,2,1].
Does production move to the alternative line?	Yes. The product no longer returns to Line 1 for station 3; it stays on Line 2 for stations 2, 3, 4 and 5.
Where does the change occur?	**Station 3** (Line 2 is kept instead of transferring back to Line 1). Station 4 is then reached by staying (26) rather than by the earlier transfer.
Effect on minimum time	38 → **39** (+1) although the machine lost 12 time units: the slow machine is bypassed, so only a small penalty is paid.

Why the new route is optimal. At station 3 the Line-1 options cost 33 (stay) or 32 (transfer from Line 2), but Line 2's own station 3 costs only 22 (stay). Being on Line 1 at station 3 costs 32 against 22 on Line 2 (10 worse), so the DP stays on Line 2 through station 5. Only at station 6 does a transfer back to Line 1 pay off, because that station takes 4 on Line 1 against 7 on Line 2: 36 + 3 = 39 against 38 + 2 = 40 for finishing on Line 2. The DP compares all $2^6 = 64$ routes implicitly and the brute-force check confirms that 39 is the true minimum.

Task 4.5 - Extension to three assembly lines (explanation only)

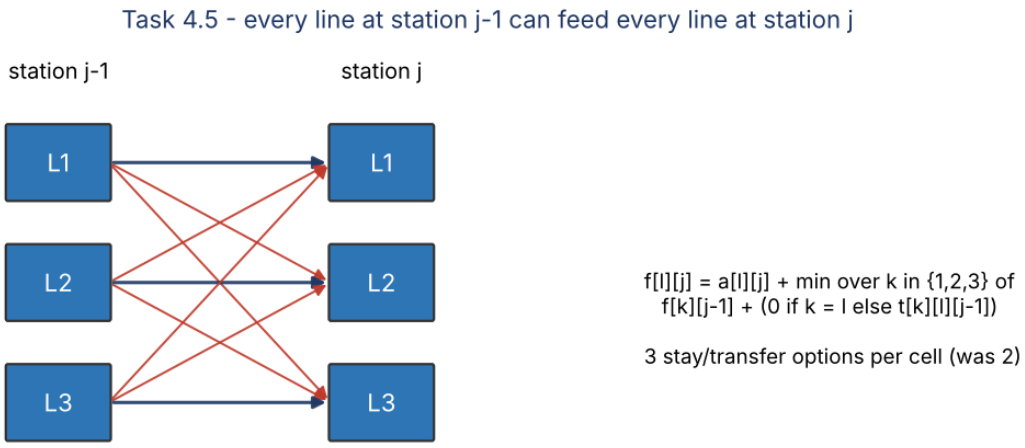


Figure 4.4 - With three lines each station-j state can be reached from any of the three states at station j−1

Component	Two lines (implemented)	Three lines (what must change)
DP state	f_l[j], l ∈ {1, 2}: 2n states	f_l[j], l ∈ {1, 2, 3}: 3n states. Still linear in n: the state is (station, line) and nothing else; the number of lines only multiplies the state count.
Transition logic	Two candidates: stay, or transfer from the one other line	Three candidates: f_l[j] = a_l[j] + min over k ∈ {1,2,3} of (f_k[j−1] + (0 if k = l, else t_{k→l}[j−1])). Two transfer options per cell instead of one: O(m²n) for m lines.
Transfer-time data	One array t_l[j] per line (because the 'other' line is unique)	Needs a **pair** (from, to): t_{k→l}[j]. A single array per line is no longer enough because it must say WHERE the product is going. If a transfer through a third line could be cheaper than a direct one, pre-compute the cheapest transfer cost for each pair first (all-pairs shortest path on the 3 lines) so the DP remains correct.
Entry / exit	e_1, e_2, x_1, x_2	e_1..e_3 and x_1..x_3; final answer = min over three last-station values.
Route reconstruction	source_l[j] is effectively a boolean (stay / other line)	source_l[j] must store the **line index** k ∈ {1,2,3} it came from, because "the other line" is ambiguous. Backtracking is the same loop: pick the best last line, then repeatedly jump to source_l[j] while decreasing j.
Trace table	Stay vs 1 transfer	Stay vs 2 transfers (3 columns per line).

The key insight is that the optimisation model is unchanged: the best way to finish station j on line l depends only on the best ways to finish station j−1 on each line (optimal substructure with only "line" as extra memory). That is why the same idea scales to m lines at O(m²n) time.

TASK 5 - Three Greedy Engineers, One Network: Kruskal, Prim, Borůvka
Graph used for all three algorithms (7 facilities, 11 undirected edges, distinct weights):

Edge	A-B	A-D	B-C	B-D	B-E	C-E	D-E	D-F	E-F	E-G	F-G
Cost	7	4	12	9	10	5	15	6	8	13	11

Source code - task5_mst.py (Kruskal, Prim, Borůvka, brute-force MST counter)

```
"""Task 5 - Kruskal, Prim, Boruvka on the same graph, with construction traces."""
import heapq, itertools

V = list("ABCDEFG")
EDGES = [("A","B",7),("A","D",4),("B","C",12),("B","D",9),("B","E",10),("C","E",5),
         ("D","E",15),("D","F",6),("E","F",8),("E","G",13),("F","G",11)]
EDGES_TIES = [(u, v, 7 if (u, v) == ("B", "D") else w) for u, v, w in EDGES]   # BD: 9 -> 7 (ties with AB)

class DSU:
    def __init__(s, items): s.p = {i: i for i in items}
    def find(s, x):
        while s.p[x] != x: s.p[x] = s.p[s.p[x]]; x = s.p[x]
        return x
    def union(s, a, b):
        ra, rb = s.find(a), s.find(b)
        if ra == rb: return False
        s.p[ra] = rb; return True

def key(e, idx, tie):              # tie = 'first' -> lower input index wins, 'last' -> higher index wins
    return (e[2], idx if tie == "first" else -idx)

def kruskal(edges, tie="first", verbose=False):
    order = sorted(range(len(edges)), key=lambda i: key(edges[i], i, tie))
    dsu, mst, total = DSU(V), [], 0
    for i in order:
        u, v, w = edges[i]
        ok = dsu.union(u, v)
        if verbose: print(f"  consider {u}-{v} ({w}): {'ACCEPT' if ok else 'reject (same component -> cycle)'}")
        if ok: mst.append((u, v, w)); total += w
        if len(mst) == len(V) - 1: break
    return mst, total

def prim(edges, start="A", tie="first", verbose=False):
    adj = {v: [] for v in V}
    for i, (u, v, w) in enumerate(edges):
        k = key((u, v, w), i, tie)
        adj[u].append((k, u, v, w)); adj[v].append((k, v, u, w))
    seen, heap, mst, total = {start}, list(adj[start]), [], 0
    heapq.heapify(heap)
    if verbose: print(f"  start at {start}; tree = {{{start}}}")
    while heap and len(seen) < len(V):
        k, u, v, w = heapq.heappop(heap)
        if v in seen: continue
        seen.add(v); mst.append((u, v, w)); total += w
        if verbose: print(f"  add {u}-{v} ({w}); tree = {{{','.join(sorted(seen))}}}")
        for item in adj[v]:
            if item[2] not in seen: heapq.heappush(heap, item)
    return mst, total

def boruvka(edges, tie="first", verbose=False):
    dsu, mst, total, rnd = DSU(V), [], 0, 0
```

```
    while len(mst) < len(V) - 1:
        rnd += 1
        cheapest = {}
        for i, (u, v, w) in enumerate(edges):
            ru, rv = dsu.find(u), dsu.find(v)
            if ru == rv: continue
            for r in (ru, rv):
                if r not in cheapest or key(edges[i], i, tie) < key(edges[cheapest[r]], cheapest[r], tie):
                    cheapest[r] = i
        if not cheapest: raise ValueError("graph is disconnected")
        if verbose:
            print(f"  Round {rnd}: components = { {r: sorted(x for x in V if dsu.find(x)==r) for r in
set(dsu.find(x) for x in V)} }")
            for r, i in sorted(cheapest.items()): print(f"      cheapest edge leaving comp[{r}] =
{edges[i][0]}-{edges[i][1]} ({edges[i][2]})")
        for i in set(cheapest.values()):
            u, v, w = edges[i]
            if dsu.union(u, v):             # union check prevents a cycle when ties pick edges in a loop
                mst.append((u, v, w)); total += w
                if verbose: print(f"      merge via {u}-{v} ({w})")
    return mst, total

def fmt(m): return sorted("".join(sorted((u, v))) + f"({w})" for u, v, w in m)

def count_msts(edges):
    best, ct, trees = None, 0, []
    for combo in itertools.combinations(range(len(edges)), len(V) - 1):
        d = DSU(V)
        if all(d.union(edges[i][0], edges[i][1]) for i in combo):
            w = sum(edges[i][2] for i in combo)
            trees.append((w, combo))
    mn = min(w for w, _ in trees)
    return mn, [c for w, c in trees if w == mn]

if __name__ == "__main__":
    print("Graph edges:", EDGES)
    for name, fn in (("KRUSKAL", kruskal), ("PRIM (start A)", prim), ("BORUVKA", boruvka)):
        print(f"\n===== {name} on distinct-weight graph =====")
        m, t = fn(EDGES, verbose=True)
        print("  MST edges:", fmt(m), " total =", t)
    mn, best = count_msts(EDGES)
    print("\nBrute force over all spanning trees: min =", mn, ", number of MSTs =", len(best))

    print("\n\n===== STRESS TEST: edge B-D changed 9 -> 7 (ties with A-B = 7) =====")
    mn, best = count_msts(EDGES_TIES)
    print("Brute force: minimum cost =", mn, ", number of distinct MSTs =", len(best))
    for c in best: print("    ", fmt([EDGES_TIES[i] for i in c]))
    print()
    for name, fn in (("Kruskal", kruskal), ("Prim", prim), ("Boruvka", boruvka)):
        for tie in ("first", "last"):
            m, t = fn(EDGES_TIES, tie=tie)
            print(f"  {name:8s} tie-break={tie:5s}: {fmt(m)} total={t}")
    print("\nTrace of Kruskal / Prim / Boruvka on tied graph (tie-break = last):")
    for name, fn in (("Kruskal", kruskal), ("Prim", prim), ("Boruvka", boruvka)):
        print(f" -- {name}"); fn(EDGES_TIES, tie="last", verbose=True)

    print("\n===== Boruvka safety test: equal-weight triangle (all edges 1) =====")
    V[:] = list("XYZ")
    tri = [("X","Y",1),("Y","Z",1),("Z","X",1)]
    print("  with union-find check:", fmt(boruvka(tri)[0]), "(2 edges, no cycle)")
    # what would happen WITHOUT the check: each component picks a different edge -> 3 edges =
cycle
```

```
    print("  without the check: X picks XY, Y picks YZ, Z picks ZX -> 3 edges for 3 vertices =
CYCLE")
    V[:] = list("ABCDEFG")
```

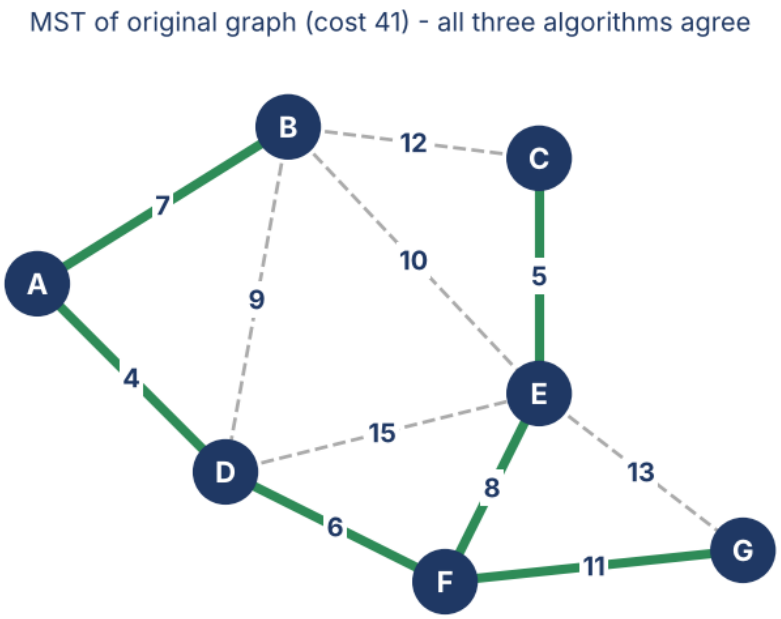


Figure 5.1 - The MST (green) found by all three algorithms: cost 41

Edge-selection traces

Kruskal - edges are examined in globally sorted order; a union-find structure tells whether the endpoints are already connected.

Order	Edge (cost)	Decision	Reason
1	A-D (4)	**Accept**	joins two separate components
2	C-E (5)	**Accept**	
3	D-F (6)	**Accept**	
4	A-B (7)	**Accept**	
5	E-F (8)	**Accept**	merges {C,E} with {A,B,D,F}
6	B-D (9)	Reject	B and D already connected → cycle
7	B-E (10)	Reject	cycle
8	F-G (11)	**Accept**	adds G; 6 edges = V−1, stop

Prim (start A) - one tree grows; the safe edge is the cheapest edge leaving the tree (a min-heap holds the frontier).

Step	Edge added (cost)	Tree after the step
1	A-D (4)	{A, D}
2	D-F (6)	{A, D, F}
3	A-B (7)	{A, B, D, F}
4	F-E (8)	{A, B, D, E, F}

Step	Edge added (cost)	Tree after the step
5	E-C (5)	{A, B, C, D, E, F}
6	F-G (11)	{A, B, C, D, E, F, G}

Note step 5: C-E costs 5, cheaper than earlier edges, yet it is added late - Prim may only choose edges that touch the current tree, whereas Kruskal accepted C-E at order 2.

Borůvka - every component simultaneously picks its cheapest outgoing edge, then all picked edges are merged.

Round	Components at start	Cheapest edge chosen by each component	Edges merged
1	7 singletons {A}…{G}	A: A-D(4); B: A-B(7); C: C-E(5); D: A-D(4); E: C-E(5); F: D-F(6); G: F-G(11)	A-D, A-B, C-E, D-F, F-G (5 distinct edges)
2	{C,E} and {A,B,D,F,G}	both choose E-F (8)	E-F

Components fell from 7 → 2 → 1: at most $\log_2 V$ rounds are ever needed because each component merges with at least one other, so the number of components at least halves per round.

	Kruskal	Prim	Borůvka
Next safe edge	Globally cheapest edge that does not close a cycle	Cheapest edge leaving the single growing tree (cut = tree vs rest)	For every component, the cheapest edge leaving it (cut property applied to all components at once)
Information maintained	Sorted edge list + union-find (component id of each vertex)	Visited set + priority queue of frontier edges (or key[v] array)	Component label per vertex + cheapest-edge array per component, rebuilt each round
How the partial solution grows	A **forest** of many fragments that merge	**One tree** that gains one vertex per step	Many fragments **merge in parallel**; component count at least halves per round
Final MST edges	A-D, C-E, D-F, A-B, E-F, F-G	A-D, D-F, A-B, E-F, C-E, F-G	A-D, A-B, C-E, D-F, F-G, E-F
Total cost	41	41	41

Stress test - multiple MSTs of equal cost

Edge **B-D** is changed from 9 to **7**, tying with A-B (7). A-B, B-D and A-D (4) form a triangle, so exactly one of A-B / B-D can join the MST. A brute-force enumeration of all C(11,6) = 462 candidate edge sets found minimum cost 41 and **exactly two MSTs**:

MST	Edges	Cost
MST #1	A-D(4), C-E(5), D-F(6), **A-B(7)**, E-F(8), F-G(11)	41
MST #2	A-D(4), C-E(5), D-F(6), **B-D(7)**, E-F(8), F-G(11)	41

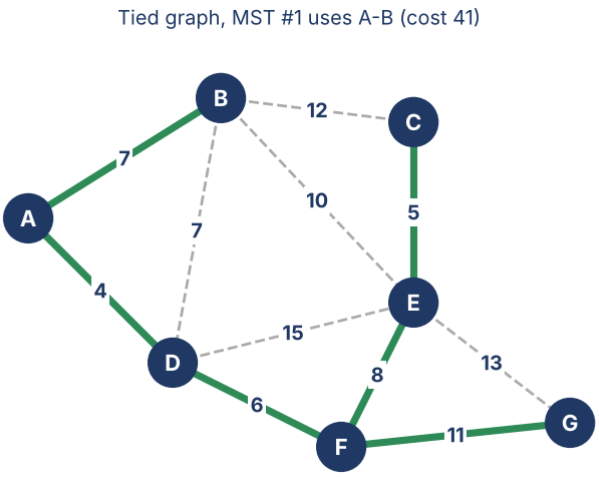


Figure 5.2a - MST #1 (uses A-B)

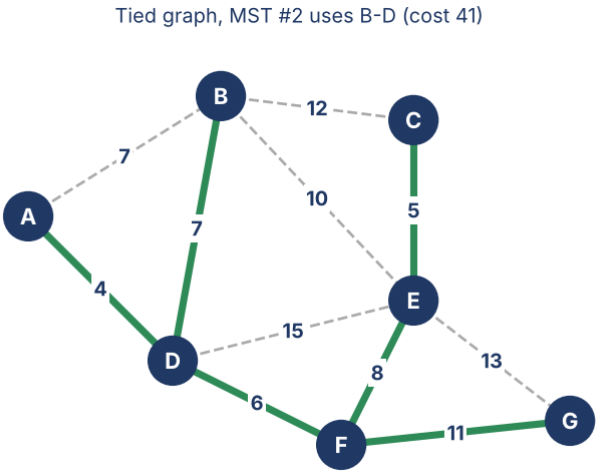


Figure 5.2b - MST #2 (uses B-D)

Algorithm	Tie-break: lower input index first	Tie-break: higher input index first
Kruskal	A-B chosen → MST #1, cost 41	B-D chosen → MST #2, cost 41
Prim	A-B chosen → MST #1, cost 41	B-D chosen → MST #2, cost 41
Borůvka	A-B chosen → MST #1, cost 41	B-D chosen → MST #2, cost 41

Observation. Yes - the algorithms can return **different edge sets while all remain correct**. Correctness of an MST algorithm only promises minimum *total cost*; the cut and cycle properties say a lightest edge is *safe*, not that it is the only safe edge. With ties, which of several equal-weight safe edges is taken depends on the tie-break rule (sort stability, heap order, scan order), so even the same algorithm can give different trees, and two different algorithms can differ. Cost stayed 41 in every run. Borůvka needs extra care: if tied components each select a different equal-weight edge, they can form a cycle (three vertices X, Y, Z with all edges 1: X picks XY, Y picks YZ, Z picks ZX). The implementation avoids this with a union-find check before merging (it returned 2 edges), or alternatively by a strict total order on edges (weight, then index).

Industry decision - which algorithm for which environment?

Environment	Characteristics that matter	Recommended	Reasoning tied to behaviour and data structures
A. Sparse network ($E \approx V$)	Few edges; edge list or adjacency list; sorting E items is cheap	**Kruskal** (Prim with a binary heap is a close second)	Kruskal sorts only E edges and then does near-constant-time union-find operations; it often stops early once $V-1$ edges are accepted, and needs no adjacency structure. Prim's heap also does well, but must build adjacency lists first.
B. Dense network ($E \approx V^2$)	Almost every pair connected; edge list is huge; adjacency matrix natural	**Prim** (array-based, $O(V^2)$)	Kruskal must sort $\sim V^2$ edges ($E \log E$). Prim with a simple key[] array scans V vertices per step and never sorts edges or needs a heap; it touches each matrix cell once and has excellent memory locality.
C. Very large network on parallel / distributed resources	Graph partitioned across machines; need independent work units	**Borůvka** (or Borůvka-style hybrid)	Its per-component cheapest-edge search is independent, so every machine can scan its own edges at the same time, and the component count at least halves each round ($7 \rightarrow 2 \rightarrow 1$ in our trace), giving $O(\log V)$ synchronisation rounds. Prim is inherently sequential (each step depends on the tree built by the previous step) and Kruskal needs a global sorted order and a shared union-find where each decision depends on earlier ones.

Caveat: in practice the choice also depends on whether the graph fits in memory and how the weights are distributed; the table follows from how each algorithm builds its solution and what data structure it must maintain, not from memorised big-O alone.

TASK 6 - Last-Mile Delivery Route Optimisation: TSP with Dynamic Programming

Task 6.1 - Model

Element	Definition
Input	Hub (index 0) and n delivery locations 1..n; cost matrix $c[i][j] \geq 0$ for every ordered pair.
Output	A tour Hub $\rightarrow \pi(1) \rightarrow \ldots \rightarrow \pi(n) \rightarrow$ Hub and its total cost.
Objective	Minimise the sum of the $n + 1$ leg costs of the tour.
Constraints	Start at the hub; visit every delivery location exactly once; return to the hub.

What must be retained at an intermediate stage? Not just the current location. Consider two partial routes that both end at location C: Hub→A→B→C and Hub→D→E→C. Their costs may be equal, but they are **not interchangeable**: the first still has to visit D and E (and F), the second still has to visit A and B. The future depends on **which set of locations has been visited** and **where we**

are now. So the state must be (S, j): the visited set and the current location (the cost so far is the value stored, and the order inside S does not matter for the future).

Task 6.2 - DP state, base, recurrence

- **State:** DP[S][j], S ⊆ {1..n} is the set of visited delivery locations, j ∈ S is the current location.
- **Stores:** the minimum cost of a path that starts at the hub, visits exactly the locations in S once each, and ends at j.
- **Base state:** DP[{j}][j] = c[Hub][j] (go directly from the hub to j).
- **Building a larger state:** DP[S][j] = min over i ∈ S \ {j} of (DP[S \ {j}][i] + c[i][j]) - the best path to j last arrived from some i, and the part before that must itself be optimal for S \ {j}.
- **Return to hub:** Answer = min over j of (DP[{1..n}][j] + c[j][Hub]).
- **Size:** n·2^(n−1) states (each j with every subset containing it) and O(n²·2^n) time. Subsets are encoded as bit masks in the code.

Task 6.3 - Implementation

Input cost matrix (Hub + 6 delivery locations A–F):

	Hub	A	B	C	D	E	F
Hub	0	12	10	19	8	15	11
A	12	0	3	7	2	9	14
B	10	3	0	6	20	5	13
C	19	7	6	0	4	16	18
D	8	2	20	4	0	8	7
E	15	9	5	16	8	0	12
F	11	14	13	18	7	12	0

Source code - task6_tsp.py

```
"""Task 6 - Travelling Salesperson (Held-Karp Dynamic Programming)."""
import itertools, random, time, tracemalloc
INF = float("inf")

def held_karp(cost):
    """cost[0] = hub; cost[1..n] = delivery locations.  Returns (best, route(index list), stats)."""
    n = len(cost) - 1
    if n < 1: raise ValueError("need at least one delivery location")
    FULL = (1 << n) - 1
    # dp[S][j]: cheapest cost starting at hub, visiting exactly the set S (bitmask over locations 1..n),
ending at j in S
    dp = [[INF] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]
    states = transitions = 0
    for j in range(n):                              # base: S = {j}
        dp[1 << j][j] = cost[0][j + 1]; states += 1
    for S in range(1, 1 << n):
        for j in range(n):
            if not (S >> j) & 1 or dp[S][j] == INF: continue
            for k in range(n):                      # extend the tour to a new location k
                if (S >> k) & 1: continue
                T = S | (1 << k); transitions += 1
```

```
            c = dp[S][j] + cost[j + 1][k + 1]
            if dp[T][k] == INF: states += 1
            if c < dp[T][k]: dp[T][k], parent[T][k] = c, j
    best, last = min((dp[FULL][j] + cost[j + 1][0], j) for j in range(n))
    route, S, j = [], FULL, last
    while j != -1:                              # backtrack through parent pointers
        route.append(j + 1); pj = parent[S][j]; S ^= (1 << j); j = pj
    route.reverse()
    return best, [0] + route + [0], {"states": states, "transitions": transitions, "table  entries": (1 << n)
* n}

def brute  force(cost):
    n = len(cost) - 1; res = []
    for p in itertools.permutations(range(1, n + 1)):
        r = [0] + list(p) + [0]
        res.append((sum(cost[a][b] for a, b in zip(r, r[1:])), r))
    return sorted(res)

NAMES = ["Hub", "A", "B", "C", "D", "E", "F"]
COST = [[0,12,10,19, 8,15,11],
        [12,0, 3, 7, 2, 9,14],
        [10,3, 0, 6,20, 5,13],
        [19,7, 6, 0, 4,16,18],
        [8, 2,20, 4, 0, 8, 7],
        [15,9, 5,16, 8, 0,12],
        [11,14,13,18,7,12, 0]]

def show(route, names): return " -> ".join(names[i] for i in route)

if    name    == "    main    ":
    print("Cost matrix (Hub + 6 delivery locations):")
    print("       " + " ".join(f"{h:>4s}" for h in NAMES))
    for i, row in enumerate(COST): print(f"{NAMES[i]:>5s} " + " ".join(f"{x:>4d}" for x in row))
    best, route, st = held  karp(COST)
    print("\nMinimum travel cost =", best)
    print("Route:", show(route, NAMES))
    print("Leg costs:", [f"{NAMES[a]}->{NAMES[b]}={COST[a][b]}" for a, b in zip(route,
route[1:])])
    print("Stats:", st)

    print("\n===== 6.4 VALIDATION: Hub + 4 locations (A-D) =====")
    sub = [row[:5] for row in COST[:5]]
    for c, r in brute  force(sub): print(f"  {show(r, NAMES):28s} cost = {c}")
    bf = brute_force(sub)[0]
    b2, r2, _ = held_karp(sub)
    print("Exhaustive optimum:", bf[0], show(bf[1], NAMES))
    print("DP result         :", b2, show(r2, NAMES), "| equal cost:", bf[0] == b2)
    # randomized cross-check
    random.seed(11); ok = 0
    for    in range(200):
        m = 4 + random.randint(0, 2)
        mat = [[0 if i == j else random.randint(1, 50) for j in range(m + 1)] for i in range(m + 1)]
        ok += brute  force(mat)[0][0] == held  karp(mat)[0]
    print(f"Randomised cross-check (asymmetric matrices, 4-6 locations): {ok}/200 agree")

    print("\n===== 6.5 STATE SPACE GROWTH =====")
    print(" n | 2^n  | DP states | transitions | n! (brute force) | time(s) | peak memory (MB)")
    import math
    random.seed(5)
    for n in range(4, 16):
        mat = [[0 if i == j else random.randint(1, 100) for j in range(n + 1)] for i in range(n + 1)]
        tracemalloc.start(); t = time.perf  counter()
        _, _, s = held_karp(mat)
```

```
        dt = time.perf_counter() - t; _, peak = tracemalloc.get_traced_memory(); tracemalloc.stop()
        print(f"{n:2d} | {2**n:5d} | {s['states']:9d} | {s['transitions']:11d} | {math.factorial(n):16d} | {dt:7.3f} | {peak/1e6:8.2f}")
    print("\nExtrapolation: n=20 -> states ~ 20*2^19 =", 20 * 2**19, "; n=30 ->", 30 * 2**29, "; n=200 ->", f"{200 * 2**199:.2e}")
    print("Memory for n=30 at 8 bytes/entry ~", f"{30 * 2**30 * 8 / 1e9:.0f} GB (table of 2^30 x 30 entries)")
```

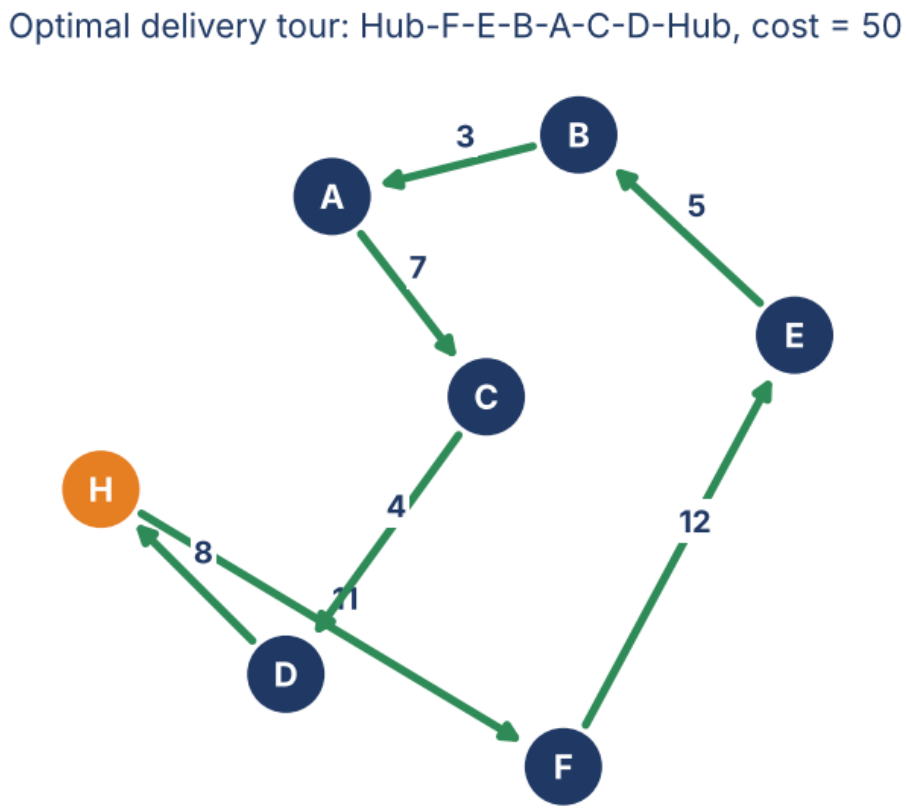


Figure 6.1 - Optimal tour

Program output

```
Minimum travel cost = 50
Route: Hub -> F -> E -> B -> A -> C -> D -> Hub
Leg costs: ['Hub->F=11', 'F->E=12', 'E->B=5', 'B->A=3', 'A->C=7', 'C->D=4', 'D->Hub=8']
Stats: {'states': 192, 'transitions': 480, 'table_entries': 384}
```

Leg	Hub→F	F→E	E→B	B→A	A→C	C→D	D→Hub	Total
Cost	11	12	5	3	7	4	8	**50**

Route: Hub → F → E → B → A → C → D → Hub, Minimum Travel Cost = 50. The DP evaluated 192 states (= $6 \cdot 2^5$) and 480 transitions.

Task 6.4 - Validation against exhaustive enumeration

Smaller instance: Hub + 4 locations (A–D, the top-left 5×5 block of the matrix). All 4! = 24 routes were enumerated independently:

Route	Cost	Route	Cost
Hub -> B -> A -> C -> D -> Hub	32	Hub -> A -> C -> B -> D -> Hub	53

Route	Cost	Route	Cost
Hub -> D -> C -> A -> B -> Hub	32	Hub -> A -> C -> D -> B -> Hub	53
Hub -> A -> B -> C -> D -> Hub	33	Hub -> B -> D -> C -> A -> Hub	53
Hub -> B -> C -> A -> D -> Hub	33	Hub -> D -> B -> C -> A -> Hub	53
Hub -> D -> A -> C -> B -> Hub	33	Hub -> C -> A -> B -> D -> Hub	57
Hub -> D -> C -> B -> A -> Hub	33	Hub -> D -> B -> A -> C -> Hub	57
Hub -> A -> D -> C -> B -> Hub	34	Hub -> A -> B -> D -> C -> Hub	58
Hub -> B -> C -> D -> A -> Hub	34	Hub -> B -> D -> A -> C -> Hub	58
Hub -> B -> A -> D -> C -> Hub	38	Hub -> C -> A -> D -> B -> Hub	58
Hub -> C -> B -> A -> D -> Hub	38	Hub -> C -> D -> B -> A -> Hub	58
Hub -> C -> D -> A -> B -> Hub	38	Hub -> A -> D -> B -> C -> Hub	59
Hub -> D -> A -> B -> C -> Hub	38	Hub -> C -> B -> D -> A -> Hub	59

Exhaustive optimum = 32 (Hub→B→A→C→D→Hub and its reverse Hub→D→C→A→B→Hub, because the matrix is symmetric). **DP result = 32** (route Hub→D→C→A→B→Hub). The costs match, and the route chosen is one of the two optimal ones. An additional randomised check on 200 **asymmetric** matrices (4–6 locations) gave 200 / 200 agreements.

How validation increases confidence: the exhaustive search uses no subproblem sharing, no bit masks and no recurrence, so it can only agree with the DP if the state definition, base case, transition and return-to-hub term are all correct (a missing return leg or a wrong mask would give a different number). Agreement on many different matrices, including asymmetric ones, makes an accidental match very unlikely. It does not prove the code for n = 15, but the logic is the same.

Task 6.5 - State-space growth

Locations n	Subsets 2^n	DP states evaluated	Transitions	Possible routes n!	Time (s)	Peak memory (MB)
4	16	32	48	24	0.000	0.00
5	32	80	160	120	0.000	0.00
6	64	192	480	720	0.001	0.01
7	128	448	1,344	5,040	0.002	0.03
8	256	1,024	3,584	40,320	0.007	0.06
9	512	2,304	9,216	362,880	0.027	0.14
10	1,024	5,120	23,040	3,628,800	0.067	0.29
11	2,048	11,264	56,320	39,916,800	0.172	0.63
12	4,096	24,576	135,168	479,001,600	0.478	1.31
13	8,192	53,248	319,488	6,227,020,800	1.101	2.75
14	16,384	114,688	745,472	87,178,291,200	2.555	5.78

Locations n	Subsets 2^n	DP states evaluated	Transitions	Possible routes n!	Time (s)	Peak memory (MB)
15	32,768	245,760	1,720,320	1,307,674,368,000	5.991	12.09

(Random cost matrices, seed 5; time and memory vary by machine, so the state and transition counts are the primary evidence.)

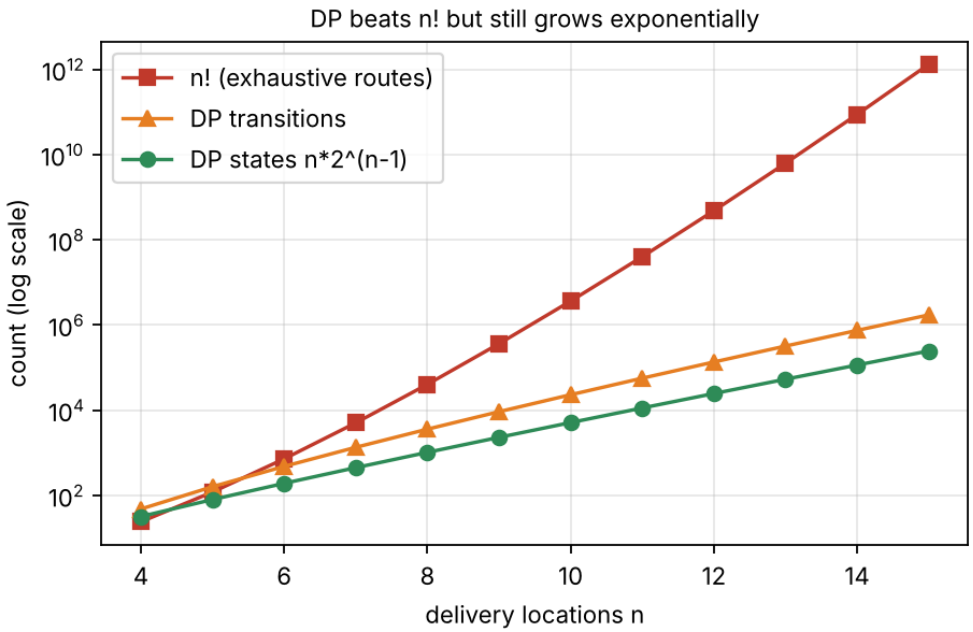


Figure 6.2 - Growth on a log scale

Why DP removes repeated work but is still expensive. Brute force recomputes the cost of the same partial route prefix in every full route that shares it; DP stores one answer per (subset, last location), so n! routes collapse to n·2^(n−1) states - at n = 15, 1.3×10^{12} routes against 245,760 states. But the number of **subsets of locations is 2^n** and every subset must be considered, so each extra location **doubles** the number of subsets: states went 5,120 (n = 10) → 11,264 → 24,576 → 53,248 → 114,688 → 245,760 (n = 15), and measured time grew from 0.07 s to about 6 s and memory from 0.29 MB to 12 MB. Extrapolating, n = 30 needs about 1.6×10^{10} states and roughly 258 GB for the table: exponential growth is reduced to a better exponential, not removed.

Task 6.6 - Business requirement change (200 locations)

- **Why exact DP is impractical:** n = 200 means $2^{200} \approx 1.6\times10^{60}$ subsets and about $200\cdot2^{199} \approx 1.6\times10^{62}$ states (roughly 10^{62}). Our own measurements already show doubling per location (n = 15 takes seconds and 12 MB), so even storing one entry per state is physically impossible, let alone computing it in a few seconds.
- **What property of the requirement changed:** the problem size and the quality target. The need moved from *guaranteed optimal for a small instance* to *good enough, within seconds, for a very large instance*; the time budget is now the binding constraint, and optimality is no longer required.
- **Strategy to investigate:** heuristic / approximation algorithms - construct a tour with nearest-neighbour or a greedy insertion heuristic (or a minimum-spanning-tree based 2-approximation when the triangle inequality holds), then improve it with local search such as 2-opt / Or-opt or

Lin-Kernighan; metaheuristics (simulated annealing, genetic algorithms) are further options. Typically $O(n^2)$ per improvement pass, which is trivial for n = 200.

- **Trade-off accepted:** the company gives up the **guarantee of optimality** (route may be a few percent longer than the best possible, with no certainty how much) and sometimes determinism/reproducibility (randomised methods), in exchange for answers in seconds that scale to hundreds or thousands of stops.

Task 6.7 - Technical reflection (≤ 150 words)

> **Answer** Held-Karp cuts TSP from n! routes to n·2^(n−1) states, but that is still exponential. My measurements show it: at n = 10 the DP evaluated 5,120 states in 0.07 s, while at n = 15 it evaluated 245,760 states, took about 6 s and used about 12 MB - every extra location roughly doubles work and memory. Extrapolated, n = 30 needs over 16 billion states. DP is scalable only when the number of distinct subproblems stays small, like knapsack's n×C table or assembly-line's 2n states. In TSP a subproblem is identified by a subset of locations, and there are 2^n subsets, so overlapping subproblems exist but are far too numerous. Being better than brute force is therefore not the same as being scalable.

Word count: 124

Summary of key results and how to reproduce

Task	Key result	Boundary / failure evidence
1	W = 20: penalty 6, 4 lines; W = 14/25/32: 51 / 9 / 4	Exact-fit words, oversize word, single word, empty input; penalty is not monotonic in W
2	Normal: greedy = DP = 250. Counterexample: greedy 30 vs DP 48	Worst-case family: greedy at 0.2 % of optimum; DP cells grow with capacity
3	Dijkstra wrong for D, F, G after E→D (−5); Bellman-Ford: D 6, F 8, G 12	Negative cycle D→F→G→E→D (−6) detected; unreachable cycle not reported
4	Min time 38; after changes 39 and 39 (route moves to Line 2)	1-station line, ties, unequal line lengths; brute-force agreement
5	All three give MST cost 41; with ties two MSTs of cost 41 exist	Borůvka cycle risk with equal weights handled by union-find
6	Optimal tour Hub-F-E-B-A-C-D-Hub, cost 50; validation 32 = 32	State growth n·2^(n−1): 245,760 states at n = 15

Reproduce: python3 task1_wordwrap.py, task2_knapsack.py, task3_routing.py, task4_assembly.py, task5_mst.py, task6_tsp.py (Python 3.8+, standard library only; the diagrams use matplotlib and networkx). Task 6 timing runs n = 4–15 and takes about 20 seconds.

Assumptions: words are separated by exactly one space (Task 1); the knapsack capacity and weights are integers (Task 2); Dijkstra treats finalised vertices as closed (Task 3); transfer times t_l[j] are paid when leaving line l after station j, and entry/exit times are added once (Task 4); the cost matrix is complete and non-negative (Task 6); timings are machine-dependent so counted operations are used as the main evidence.